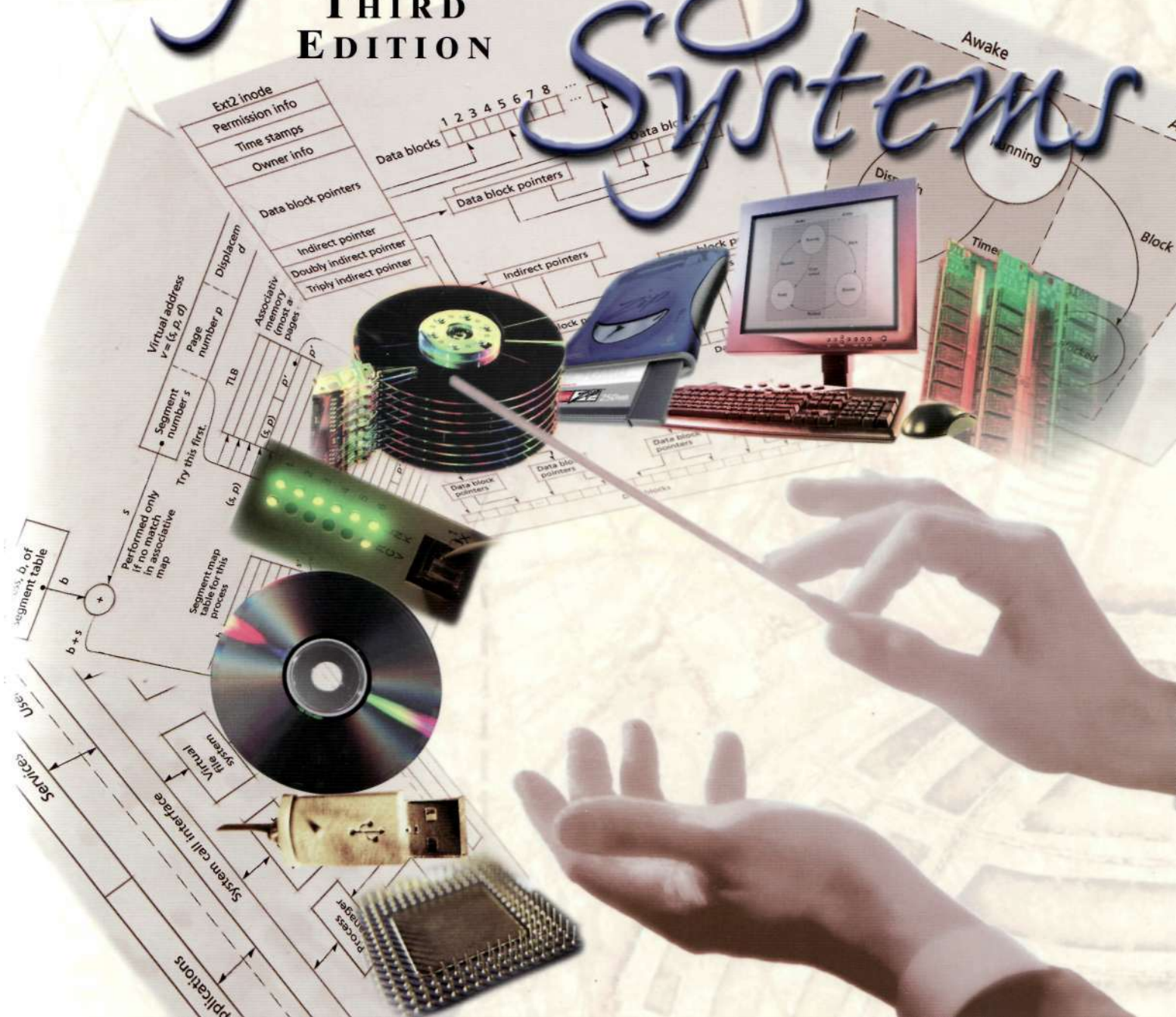


CHOFFNES

**THIRD
EDITION**



Operating Systems

Third Edition

H. M. Deitel

Deitel & Associates, Inc.

P. J. Deitel

Deitel & Associates, Inc.

D. R. Choffnes

Deitel & Associates, Inc.



Upper Saddle River, NJ 07458

Deitel Books. Cyber Classrooms, Complete Training Courses and Web-Based Training Courses published by Prentice Hall

Computer Science Series

Operating Systems, 3/E

How To Program Series

Advanced Java™ 2 Platform How to Program

C How to Program, 4/E

C++ How to Program, 4/E

C# ® How to Program

e-Business and e-Commerce How to Program

Internet and World Wide Web How to Program, 3/E

Java™ How to Program, 5/E

Perl How to Program

Python How to Program

Visual Basic® 6 How to Program

Visual Basic® .NET How to Program, 2/E

Visual C++® .NET How to Program

Wireless Internet & Mobile Business How to Program

XML How to Program

Simply Series

Simply C#®:An Application-Driven Tutorial Approach

Simply Java™ Programming: An Application-Driven Tutorial Approach

Simply Visual Basic® .NET: An Application-Driven Tutorial Approach (Visual Studio .NET2002 Edition)

Simply Visual Basic® .NET: An Application-Driven Tutorial Approach (Visual Studio .NET2003 Edition)

CS1 Programming Series

Java™ Software Design

.NET How to Program Series

C#® How to Program

Visual Basic® .NET How to Program, 2/E

Visual C++® .NET How to Program

Visual Studio® Series

C#® How to Program

Getting Started with Microsoft® Visual C++® 6 with an Introduction to MFC

Simply C#®:An Application-Driven Tutorial Approach

Simply Visual Basic® .NET: An Application-Driven Tutorial Approach (Visual Studio .NET2002 Edition)

Simply Visual Basic® .NET: An Application-Driven Tutorial Approach (Visual Studio .NET 2003 Edition)

Visual Basic® 6 How to Program

Visual Basic® .NET How to Program, 2/E

Visual C++® .NET How to Program

Deitel® Developer Series

Java™ Web Services for Experienced Programmers

Web Services A Technical Introduction

For Managers Series

e-Business and e-Commerce for Managers

Interactive Web-Based Training Series

Premium CourseCompass Version of Visual Basic® .NET Multimedia Cyber Classroom, 2/E

Premium CourseCompass Version of Java™ 2 Multimedia Cyber Classroom, 5/E

Premium CourseCompass Version of C++ Multimedia Cyber Classroom, 4/E

The Complete Training Course Series

The Complete C++ Training Course, 4/E
The Complete C#® Training Course
*The Complete e-Business and e-Commerce
Programming Training Course*
*The Complete Internet and World Wide Web
Programming Training Course, 2/E*
The Complete Java™ 2 Training Course, 5/E
The Complete Perl Training Course
The Complete Python Training Course
*The Complete Visual Basic® 6 Training
Course*
*The Complete Visual Basic® .NET Training
Course, 2/E*
*The Complete Wireless Internet & Mobile
Business Programming Training Course*
*The Complete XML Programming Training
Course*

***Interactive Multimedia Cyber Classroom
Series***

C++ Multimedia Cyber Classroom, 4/E
C#® Multimedia Cyber Classroom
*e-Business and e-Commerce Multimedia
Cyber Classroom*
*Internet and World Wide Web Multimedia
Cyber Classroom, 2/E*
Java™2 Multimedia Cyber Classroom, 5/E
Perl Multimedia Cyber Classroom
Python Multimedia Cyber Classroom
*Visual Basic® 6 Multimedia Cyber
Classroom*
*Visual Basic® .NET Multimedia Cyber
Classroom, 2/E*
*Wireless Internet & Mobile Business
Programming Multimedia Cyber
Classroom*
XML Multimedia Cyber Classroom

To follow the Deitel publishing program, please subscribe to the free
Deitel® Buzz Online e-mail newsletter at:

www.deitel.com/newsletter/subscribe.html

To communicate with the authors, send e-mail to:

deitel@deitel.com

For information on Deitel instructor-led, corporate training seminars offered
worldwide visit:

www.deitel.com

For continuing updates on Prentice Hall and Deitel publications visit:

www.deitel.com,
www.prenhall.com/deitel or
www.InformIT.com/deitel

Vice President and Editorial Director, ECS: **Marcia J. Horton**
Senior Acquisitions Editor: **Kate Hargett**
Associate Editor: **Jennifer Cappello**
Assistant Editor: **Sarah Parker**
Editorial Assistant: **Michael Giacobbe**
Vice President and Director of Production and Manufacturing, ESM: **David W. Riccardi**
Executive Managing Editor: **Vince O'Brien**
Managing Editor: **Tom Manshreck**
Production Editors: **Chirag Thakkar, John F. Loveli**
Production Editor, Media: **Bob Engelhardt**
Production Assistant: **Daniela Petrilli**
Director of Creative Services: **Paul Belfanti**
A/V Production Editor: **Xiaohong Zhu**
Art Studio: **Artworks, York, PA**
Creative Director: **Carole Anson**
Art Director: **Geoffrey Cassar**
Cover Design: **Harvey M. Deitel, David R. Choffnes, Suzanne Behnke**
Interior Design: **Harvey M. Deitel, David R. Choffnes, Geoffrey Cassar, Anne DeMarinis**
Manufacturing Manager: **Trudy Piscioti**
Manufacturing Buyer: **Lisa McDowell**
Marketing Manager: **Pamela Hersperger**
Marketing Assistant: **Barrie Reinhold**



© 2004 by Pearson Education, Inc.

Pearson Prentice Hall
Pearson Education, Inc.
Upper Saddle River, New Jersey 07458

All rights reserved. No part of this book may be reproduced, in any form or by any means, without permission in writing from the publisher. Pearson Prentice Hall® is a trademark of Pearson Education, Inc.

The authors and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The authors and publisher make no warranty of any kind, expressed or implied, with regard to these programs or to the documentation contained in this book. The authors and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks and registered trademarks. Where those designations appear in this book, and Pearson Education, Inc. and the authors were aware of a trademark claim, the designations have been printed in initial caps or all caps. All product names mentioned remain trademarks or registered trademarks of their respective owners.

DEITEL is a registered trademark of Deitel and Associates, Inc.

All rights reserved. No part of this book may be reproduced, in any form or by any means, without permission in writing from the publisher.

If you purchased this book within the United States or Canada you should be aware that it has been wrongfully imported without the approval of the Publisher or the Author.

Printed in the United States of America

10 9 8 7 6 5 4 3 2 1

ISBN 0-13-124696-8

Pearson Education Ltd., *London*
Pearson Education Australia Pty. Ltd., *Sydney*
Pearson Education Singapore, Pte. Ltd.
Pearson Education North Asia Ltd., *Hong Kong*
Pearson Education Canada, Inc., *Toronto*
Pearson Educacion de Mexico, S.A. de C.V.
Pearson Education-Japan, *Tokyo*
Pearson Education Malaysia, Pte. Ltd.
Pearson Education, Inc., *Upper Saddle River, New Jersey*

In memory of Edsger W. Dijkstra.

*Harvey M. Deitel
Paul J. Deitel
David R. Choffnes*

*To my grandparents, Beverly and Jay Peddy:
For your love, generosity and inspiration.*

David

Contents

<i>List of Illustrations</i>	<i>xxvii</i>
<i>Preface.....</i>	<i>xxvii</i>
 <i>Part 1 Introduction to Hardware, Software and Operating System.....</i>	 <i>1</i>
1 Introduction to Operating Systems.....	3
1.1 Introduction.....	6
1.2 What Is an Operating System?.....	6
1.3 Early History: The 1940s and 1950s.....	7
Operating Systems Thinking: Innovation.....	8
1.4 The 1960s.....	9
Operating Systems Thinking: Relative Value of Human and Computer Resources.....	10
Biographical Note: Ken Thompson and Dennis Ritchie.....	12
1.5 The 1970s.....	13
Anecdote Abraham Lincoln's Technology Caution.....	14
1.6 The 1980s.....	15
1.7 History of the Internet and World Wide Web.....	16
Biographical Note: Doug Engelbart.....	16
Biographical Note: Tim Berners-lee.....	18
1.8 The 1990s.....	19
Biographical Note: Linus Torvalds.....	21
Biographical Note: Richard Stallman.....	22
1.9 2000 and Beyond.....	23
1.10 Application Bases.....	25
1.11 Operating System Environments.....	26
1.12 Operating System Components and Goals.....	28
12.1.1 Core Operating System Components.....	28

1.12.2	Operating System Goals.....	30
	Operating System Thinking: Performance.....	31
1.13	Operating System Architectures.....	33
	Operating Systems Thinking: Keep It Simple (KIS)	33
1.13.1	Monolithic Architecture.....	33
	Anecdote: System Architect vs. System Engineer.....	34
1.13.2	latyered Architecture.....	35
	Operating Systems Thinking: Architecture.....	35
1.13.3	Microkernel Architecture.....	36
1.13.4	Networked and Distributed Operating Systems.....	38
2	Hardware and Software Concepts	53
2.1	Introduction.....	56
2.2	Evolution of Hardware Devices.....	56
	Biographical Note: Gordon Moore and Moore's law.....	57
2.3	Hardware Components.....	59
23..1	Mainboard.....	59
2.3.2	Processors	60
2.3.3	Clocks.....	62
2.3.4	Memory Hierarchy	63
	Operating Systems Thinking: Caching.....	65
2.3.5	Main Memory.....	65
2.3.6	Secondary Storage.....	66
2.3.7	Buses.....	67
2.3.8	Direct Memory Access (DMA).....	68
	Operating Systems Thinking: legacy Hardware and Software.....	70
2.3.9	Peripheral Devices.....	70
2.4	Hardware Support for Operating Systems.....	72
2.4.1	Processor.....	72
	Operating Systems Thinking: Principle of least Privilege.....	73
	Operating Systems Thinking: Protection.....	74
	Anecdote: Origins of the Term "Glitch"	75
2.4.2	Timers and Clocks	76
2.4.3	Bootstrapping.....	76
24.4	Plug and Play	77

2.5	Caching and Buffering.....	78
	Operating Systems Thinking: Heuristics.....	79
2.6	Software Overview.....	80
	2.6.1 Machine language and Assembly language.....	80
	2.6.2 Interpreters and Compilers	81
	2.6.3 High-level languages.....	82
	2.6.4 Structured Programming	83
	2.6.5 Object-Oriented Programming.....	83
2.7	Application Programming Interface (APIs).....	85
2.8	Compiling, linking and loading.....	86
	2.8.1 Compiling.....	86
	2.8.2 linking.....	87
	Mini Case Study: Mach.....	90
	2.8.3 loading.....	91
2.9	Firmware.....	93
2.10	Middleware.....	93

Part 2 Processes and Threads..... 106

3 Process Concepts..... 109

3.1	Introduction..... 111
	Operating Systems Thinking: Customers Ultimately Want Applications..... 111
	3.1.1 Definition of Process
	Mini Case Study: CTSS and Multics
	Biographical Note: Fernando J. Corbato.....
3.2	Process States: Life Cycle of a Process.....113
3.3	Process Management.....114
	3.3.1 Process States and State Transitions.....115
	3.3.2 Process Control Blocks (PCBs)/Process Descriptors..... 116
	Operating Systems Thinking: Data Structures in Operating Systems..... 118
	3.3.3 Process Operations..... 118
	3.3.4 Suspend and Resume..... 120
	3.3.5 Context Switching..... 122
3.4	Interrupts..... 123
	Operating Systems Thinking: Asynchronism vs. Synchronism..... 124

3.4.1	Interrupt Processing.....	120
4.4.2	Interrupt Classes.....	127
3.5	Interprocess Communication.....	129
3.5.1	Signals.....	130
3.5.2	Message Passing	130
3.6	Case Study: UNIX Processes	132
	Mini Case Study: UNIX Systems.....	133

4 Thread Concepts.....145

4.1	Introduction.....	147
	Operating Systems Thinking: Concurrency	147
4.2	Definition of Thread.....	148
4.3	Motivation for Threads	150
	Operating Systems Thinking: Parallelism.....	151
4.4	Thread States: Life Cycle of a Thread.....	152
4.5	Thread Operations	154
4.6	Threading Models	155
4.6.1	User-level Threads.....	155
4.6.2	Kernel-level Threads.....	157
4.6.3	Combining User- and Kernel-Level Threads.....	159
	Operating Systems Thinking: Standards Conformance.....	159
4.7	Thread Implementation Considerations.....	161
4.7.1	Thread Signal Delivery.....	161
	Anecdote: Engineering.....	163
4.7.2	Thread Termination.....	165
4.8	POSIX and Pthreads.....	165
	Anecdote: Standards and Conformance: Plug-to-Plug Compatibility	166
4.9	Linux Threads.....	167
	Operating Systems Thinking: Scalability.....	168
4.10	Windows XP Threads.....	169
4.11	Java Multithreading Case Study, Part I: Introduction to Java Threads.....	172

5 Asynchronous Concurrent Execntion.....185

5.1	Introduction.....	187
-----	-------------------	-----

5.2	Mutual Exclusion.....	187
5.2.1	Java Multithreading Case Study, Part II: A Producer/Consumer Relationship in Java.....	188
5.2.2	Critical Section.....	197
5.2.3	Mutual Exclusion Primitives	198
5.3	Implementing Mutual Exclusion Primitives.....	199
5.4	Software Solutions to the Mutual Exclusion Problem.....	200
5.4.1	Dekker's Algorithm.....	200
5.4.2	Peterson's Algorithm.....	211
	Anecdote: Why Should t you Believe That your Software Is Functioning Correctly?.....	211
5.4.3	N-Thread Mutual Exclusion: Lamport's Bakery Algorithm.....	215
	Biographical Note: Leslie Lamport.....	216
5.5	Hardware Solutions to the Mutual Exclusion Problem.....	221
5.5.1	Disabling Interrupts.....	221
5.5.2	Test-and-Set Instruction.....	222
5.5.3	Swap Instruction.....	224
5.6	Semaphores.....	227
	Biographical Note: Edsger W. Dijkstra.....	227
5.6.1	Mutual Exclusion with Semaphores	228
5.6.2	Thread Synchronization with Semaphores.....	229
5.6.3	Counting Semaphores.....	231
5.6.4	Implementing Semaphores.....	232
	Anecdote: Unclear Requirements.....	233
6	Concurrent Programming.....	245
6.1	Introduction.....	247
	Anecdote, Exhaustive Testing Is Impossible.....	248
6.2	Monitors.....	249
	Operating Systems Thinking: Information Hiding	250
6.2.1	Condition Variables.....	251
6.2.2	Simple-Resource-Allocation with Monitors	252
	Biographical Note: Per Brinch Hansen.....	252
6.2.3	Monitor Example: Circular Buffer.....	254
6.2.4	Monitor Example: Readers and Writers	257
6.3	Java Monitors.....	260

6.4	Java Multithreading Case Study, Part III. Producer/Consumer Relationship in Java ...	261
6.5	Java Multithreading Case Study, Part IV Circular Buffer in Java.....	270
7	Deadlock and Indefinite Postponement.....	289
7.1	Introduction.....	291
7.2	Examples of Deadlock	291
7.2.1	Traffic Deadlock	292
7.2.2	Simple Resource Deadlock	293
	Anecdote: One-lane Bridge.....	293
7.2.3	Deadlock in Spooling Systems.....	294
	Operating Systems Thinking: Waiting, Deadlock and Indefinite Postponement.....	295
7.2.4	Example: Dining Philosophers.....	296
7.3	Related Problem: Indefinite Postponement	298
7.4	Resource Concepts.....	298
7.5	Four Necessary Conditions for Deadlock	299
7.6	Deadlock Solutions.....	300
7.7	Deadlock Prevention.....	301
7.7.1	Denying the "Wait-For" Condition.....	302
	Anecdote: No Nuts, Bolts or Screws Allowed.....	302
7.7.2	Denying the "No-Preemption" Condition.....	303
7.7.3	Denying the "Circular-Wait" Condition.....	304
7.8	Deadlock Avoidance with Dijkstra's Banker's Algorithm.....	306
	Anecdote: Acronyms	307
7.8.1	Example of a Safe States.....	309
7.8.2	Example of an Unsafe State.....	309
7.8.3	Example of Safe-State-to-Unsafe-State Transition.....	310
7.8.4	Banker's Algorithm Resource Allocation.....	311
7.8.5	Weaknesses in the Banker's Algorithm.....	312
7.9	Deadlock Detection.....	312
7.9.1	Resource-Allocation Graphs.....	313
7.9.2	Reduction of Resource-Allocation Graphs.....	314
7.10	Deadlock Recovery	316
7.11	Deadlock Strategies in Current and Future Systems.....	318

8 Processor Scheduling.....333

8.1	Introduction.....	335
8.2	Scheduling Levels.....	335
8.3	Preemptive vs. Nonpreemptive Scheduling	337
	Operating Systems Thinking: Overhead.....	338
8.4	Priorities.....	339
8.5	Scheduling Objectives	340
	Operating Systems Thinking: Predictability.....	341
	Operating Systems Thinking: Fairness.....	342
8.6	Scheduling Criteria.....	342
8.7	Scheduling Algorithms.....	344
8.7.1	First-In-First-Out (FIFO) Scheduling.....	344
8.7.2	Round-Robin (RR) Scheduling.....	345
8.7.3	Shortest-Process-First (SPF) Scheduling.....	348
8.7.4	Highest-Response-Ratio-Next (HRRN) Scheduling.....	349
8.7.5	Shortest-Remaining-Time- (SRT) Scheduling.....	349
8.7.6	Multilevel Feedback Queues	350
8.7.7	Fair Share Scheduling.....	354
8.8	Deadline Scheduling.....	356
	Operating Systems Thinking: Intensity of Resource Management vs. Relative Resource Value . . .	357
8.9	Real-Time-Scheduling.....	357
	Mini Case Study: Real-Time Operating Systems.....	358
8.10	Java Thread Scheduling.....	360

Part 3 Physical and Virtual Memory..... 374

9 Real Memory Organization and Management..... 377

9.1	Introduction.....	379
9.2	Memory Organization.....	379
	Operating Systems Thinking: There Are No Upper Limits to Processing Power, Memory, Storage and Bandwidth.....	380
9.3	Memory Management	380
9.4	Memory Hierarchy.....	382
9.5	Memory Management Strategies.....	383

9.6	Contiguous vs. Noncontiguous Memory Allocation.....	384
9.7	Single-User Contiguous Memory Allocation.....	385
9.7.1	Overlays.....	386
9.7.2	Protection in a Single-User System.....	387
9.7.3	Single-Stream Batch Processing.....	389
	Operating Systems Thinking: Change Is the Rule Rather Than the Exception.....	389
9.8	Fixed-Partition Multiprogramming.....	390
	Anecdote: Compartmentalization.....	393
	Operating Systems Thinking: Spatial Resources and Fragmentation	394
9.9	Variable-Partition Multiprogramming.....	395
9.9.1	Variable-Partition Characteristics.....	395
9.9.2	Memory Placement Strategies.....	399
9.10	Multiprogramming with Memory Swapping.....	401
10 Virtual Memory Organization.....		413
10.1	Introduction.....	415
	Mini case Study: Atlas.....	415
	Operating Systems Thinking: Virtualization.....	416
	Biographical Note: Peter Denning.....	417
10.2	Virtual Memory: Basic Concepts	418
	Anecdote: Virtual Memory Unnecessary	418
10.3	Block Mapping.....	420
10.4	Paging.....	424
10.4.1	Paging Address Translation by Direct Mapping.....	428
10.4.2	Paging Address Translation by Associative Mapping.....	429
10.4.3	Paging Address Translation with Direct/Associative Mapping.....	430
	Operating Systems Thinking: Empirical Results: locality-Based Heuristics.....	432
10.4.4	Multilevel Page Tables.....	433
10.4.5	Inverted Page Tables.....	436
10.4.6	Sharing in a Paging System.....	439
	Operating Systems Thinking: lazy Allocation.....	441
10.5	Segmentation.....	442
10.5.1	Segmentation Address Translation by Direct Mapping.....	444
10.5.2	Sharing in a Segmentation System.....	446

10.5.3	Protection and Access Control in Segmentation Systems.....	447
10.6	Segmentation Paging Systems.....	451
10.6.1	Dynamic Address Translation in a Segmentation/Paging System.....	451
10.6.2	Sharing and Protection in a Segmentation/Paging System.....	455
10.7	Case Study: IA.32 Intel Architecture Virtual Memory	455
	Mini Case Study: IBM Mainframe Operating Systems.....	459
	Mini Case Study: Early History of the VM Operating System.....	461
11	Virtual Memory Management.....	477
11.1	Introduction.....	479
11.2	Locality.....	479
11.3	Demand Paging.....	480
	Operating Systems Thinking, Computer Theory in Operating Systems.....	481
	Operating Systems Thinking, Space-Time Trade-offs.....	482
11.4	Anticipatory Paging	483
11.5	Page Replacement	485
11.6	Page-Replacement Strategies	486
11.6.1	Random Page Replacement.....	486
11.6.2	First-In-First-Out (FIFO) Page Replacement.....	487
11.6.3	FIFO Anomaly.....	488
11.6.4	Least-Recently-Used (LRU) Page Replacement.....	489
11.6.5	Least-Frequently-Used (LFU) Page-Replacement.....	491
11.6.6	Not-Used-Recently (NUR) Page Replacement.....	491
11.6.7	Modifications to FIFO: Second-Chance and Clock Page Replacement.....	493
11.6.8	Far Page Replacement.....	494
11.7	Working Set Model.....	496
11.8	Page-Fault-Frequency (PFF) Page Replacement.....	500
11.9	Page Release.....	502
11.10	Page Size.....	502
11.11	Program Behavior under Paging.....	505
11.12	Global vs. local Page Replacement.....	507
11.13	Case Study, Linux Page Replacement.....	508

<i>Part 4 Secondary Storage, Files and Databases.....</i>	<i>522</i>
<i>12 Disk Performance Otimization.....</i>	<i>525</i>
12.1 Introduction.....	528
12.2 Evolution of Secondary Storage.....	528
12.3 Characteristics of Moving.Head Disk Storage.....	529
12.4 Why Disk Scheduling Is Necessary	531
12.5 Disk Scheduling Strategies	533
12.5.1 First-Come-First-Served (FCFS) Disk Scheduling	535
12.5.2 Shortest-Seek-Time-First (SSTF) Disk Scheduling.....	536
<i>Anecdote: Every Problem Has a Solution and Every Solution Has a Problem.....</i>	<i>536</i>
12.5.3 SCAN Disk Scheduling	537
12.5.4 C-SCAN Disk Scheduling.....	538
12.5.5 FSCAN and N-Step SCAN Disk Scheduling.....	539
12.5.6 LOOK and C-LOOK Disk Scheduling.....	541
12.6 Rotational Optimization.....	542
12.6.1 SLTF Scheduling.....	542
12.6.2 SPTF and SATF Scheduling.....	543
12.7 System Considerations.....	546
<i>Operating Systems Thinking: Saturation and Bottlenecks.....</i>	<i>546</i>
12.8 Caching and Buffering.....	548
12.9 Other Disk Performances Techniques.....	549
<i>Operating Systems Thinking: Compression and Decompression.....</i>	<i>550</i>
<i>Operating Systems Thinking: Redundancy.....</i>	<i>551</i>
12.10 Redundant Arrays of Independent Disks (RAID)	552
12.10.1 RAID Overview.....	552
<i>Operating Systems Thinking: Fault Tolerance.....</i>	<i>554</i>
<i>Operating Systems Thinking: Mission-n-Critical Systems.....</i>	<i>554</i>
12.10.2 Level 0 (Striping).....	555
12.10.3 Level 1 (Mirroring).....	556
12.10.4 Level 2 (Bit-level Hamming ECC Parity).....	558
12.10.5 Level 3 (Bit-level XOR ECC Parity)	560
12.10.6 Level 4 (Block-level XOR ECC Parity).....	562
12.10.7 level 5 (Block-level Distributed XOR ECC Parity).....	563

13 File and Database Systems.....	583
13.1 Introduction.....	586
13.2 Data Hierarchy.....	586
13.3 Files.....	587
13.4 File Systems.....	589
Operating Systems Thinking: Encryption and Decryption.....	590
13.4.1 Directories	590
13.4.2 Metadata.....	594
13.4.3 Mounting.....	596
13.5 File Organization.....	598
13.6 File Allocation.....	599
13.6.1 Contiguous File Allocation.....	599
13.6.2 Linked-list Noncontiguous File Allocation.....	600
13.6.3 Tabular Noncontiguous File Allocation.....	602
Mini Case Study: MS-DOS.....	604
13.6.4 Indexed Noncontiguous File Allocation.....	605
13.7 Free Space Management.....	608
13.8 File Access Control.....	610
13.8.1 Access Control fMatrix.....	610
Operating Systems Thinking: Security.....	610
13.8.2 Access Control by User Classes.....	611
13.9 Data Access Techniques.....	612
13.10 Data Integrity Protection.....	613
13.10.1 Backup and Recovery.....	613
Operating Systems Thinking: Backup and Recovery.....	614
13.10.2 Data Integrity and log Structured File Systems.....	610
Operating Systems Thinking: Murphy's law and Robust Systems	616
13.11 File Servers and Distributed Systems.....	619
13.12 Database Systems.....	619
13.12.1 Advantages of Database Systems.....	620
13.12.2 Data Access.....	620
13.12.3 Relational Database Model.....	621
13.12.4 Operating Systems and Database Systems.....	623

<i>Part 5 Performance, Processors and Multiprocessor Management</i>	638
<i>14 Performance and Processor Design</i>	641
14.1 Introduction.....	643
14.2 Important Trends Affecting Performance Issues.....	643
14.3 Why Performance Monitoring and Evaluation Are Needed.....	644
14.4 Performance Measures	645
14.5 Performance Evaluation Techniques.....	647
14.5.1 Tracing and Profiting.....	648
14.5.2 Timings and Microbenchmarks	649
14.5.3 Application Specific Evaluation.....	650
14.5.4 Analytic Models.....	652
14.5.5 Benchmarks.....	652
14.5.6 Synthetic Programs.....	654
14.5.7 Simulation.....	655
14.5.8 Performance Monitoring.....	656
14.6 Bottlenecks and Saturation.....	658
14.7 Feedback Loops.....	659
14.7.1 Negative Feedback.....	659
14.7.2 Positive Feedback.....	659
14.8 Performance Techniques in Processor Design.....	660
14.8.1 Complex Instruction Set Computing (CISC)	661
14.8.2 Reduced Instruction Set Computing (RISC).....	662
14.8.3 Post-RISC Processors.....	665
14.8.4 Explicitly Parallel Instruction Computing (EPIC).....	668
<i>15 Multiprocessor Management</i>	683
15.1 Introduction.....	686
Mini Case Study: Supercomputers.....	686
Biographical Note: Seymour Cray	687
15.2 Multiprocessor Architecture.....	688
15.2.1 Classifying Sequential and Parallel Architectures.....	688
15.2.2 Processor Interconnection Schemes.....	690
15.2.3 Loosely Coupled vs. Tightly Coupled Systems.....	696

15.3	Multiprocessor Operating System Organizations.....	698
15.3.1	Master/Slave.....	698
15.3.2	Separate Kernels.....	699
15.3.3	Symmetrical Organization.....	700
	Operating Systems Thinking: Graceful Degradation.....	701
15.4	Memory Access Architectures.....	701
15.4.1	Uniform Memory Access.....	702
15.4.2	Nonuniform Memory Access.....	703
15.4.3	Cache-Only Memory Architecture.....	704
15.4.4	No Remote Memory Access.....	705
15.5	Multiprocessor Memory Sharing.....	707
	Operating Systems Thinking: Data Replication and Coherency.....	707
15.5.1	Cache Coherence.....	708
15.5.2	Page Replication and Migration.....	709
15.5.3	Shared Virtual Memory	710
15.6	Multiprocessor Scheduling.....	713
15.6.1	Job-Blind Multiprocessor Scheduling.....	714
15.6.2	Job-Aware Multiprocessor Scheduling.....	715
15.7	Process Migration.....	719
15.7.1	Flow of Process Migration.....	719
15.7.2	Process Migration Concepts.....	721
15.7.3	Process Migration Strategies.....	722
15.8	Load Balancing.....	723
15.8.1	Static load Balancing.....	724
15.8.2	Dynamic load Balancing.....	725
15.9	Multiprocessor Mutual Exclusion.....	729
15.9.1	Spin locks.....	729
15.9.2	Sleep/Wakeup locks	730
15.9.3	Read/Write locks.....	731

Part 6 Networking and Distributed Computing..... 746

16	Introduction to Networking.....	749
16.1	Introduction.....	751

16.2	Network Topology.....	751
	Mini Case Study: Symbian OS.....	754
16.3	Network Types	754
16.4	TCP/IP Protocol Stack.....	755
16.5	Application Layer.....	756
	16.5.1 Hypertext Transfer Protocol (HTTP).....	757
	16.5.2 File Transfer Protocol (FTP)	758
16.6	Transport Layer.....	758
	Anecdote: Unreliable Communication lines Can lead to Embarrassment.....	759
	16.6.1 Transmission Control Protocol (TCT)	760
	16.6.2 User Datagram Protocol (UDP).....	761
16.7	Network Layer.....	762
	16.7.1 Internet Protocol (IP)	763
	16.7.2 Internet Protocol version 6 (IPv6).....	763
16.8	Link Layer.....	764
	16.8.1 Ethernet.....	765
	16.8.2 Token Ring.....	766
	16.8.3 Fiber Distributed Data Interface (FDDI).....	766
	16.8.4 IEEE 802.11 (Wireless).....	768
16.9	Client/Server Model.....	768
17 Introduction to Distributed Systems.....		781
17.1	Introduction.....	783
17.2	Attributes of Distributed Systems.....	784
	17.2.1 Performance and Scalability.....	784
	17.2.2 Connectivity and Security.....	784
	17.2.3 Reliability and Fault Tolerance.....	785
	17.2.4 Transparency	786
	17.2.5 Network Operating Systems.....	787
	17.2.6 Distributed Operating Systems	787
17.3	Communication in Distributed Systems.....	788
	17.3.1 Middleware.....	788
	Anecdote: Consequences of Errors.....	789
	17.3.2 Remote Procedure Coll (RPC)	790

17.3.3	Remote Method Invocation (RMI).....	791
17.3.4	CORBA (Common Object Request Broker Architectures)	792
17.3.5	DCOM (Distributed Component Object Model).....	793
17.3.6	Process Migration in Distributed Systems.....	793
17.4	Synchronization in Distributed Systems.....	794
17.5	Mutual Exclusion Distributed Systems.....	795
17.5.1	Mutual Exclusion without Shared Memory	795
17.5.2	Agrawala and Ricart's Distributed Mutual Exclusion Algorithm.....	796
17.6	Deadlock in Distributed Systems.....	797
17.6.1	Distributed Deadlock.....	797
17.6.2	Deadlock Prevention.....	798
17.6.3	Deadlock Detection.....	800
17.6.4	A Distributed Resource Deadlock Algorithm.....	801
17.7	Case Study: The Sprite Distributed Operating System.....	803
17.8	Case Study: The Amoeba Distributed Operating System.....	804
18	Distributed System and Web Services.....	815
18.1	Introduction.....	817
18.2	Distributed File Systems	817
18.2.1	Distributed file System Concepts.....	817
18.2.2	Network File System (NFS).....	820
18.2.3	Andrew File System (AFS)	822
18.2.4	Coda File System.....	824
18.2.5	Sprite File System.....	827
18.3	Multicomputer Systems.....	830
18.4	Clustering.....	831
18.4.1	Clustering Types.....	832
18.4.2	Clustering Benefits.....	833
18.4.3	Clustering Examples.....	833
18.5	Peer-to-Peer Distributed Computing.....	836
18.5.1	Client/Server and Peer-to-Peer Applications.....	836
18.5.2	Centralized vs. Decentralized P2P Applications.....	836
18.5.3	Peer Discovery and Searching.....	839
18.5.4	JXTA.....	840
18.6	Grid Computing.....	842

18.7	Java Distributed Computing.....	843
18.7.1	Java Servlets and JavaServer Pages (JSP).....	843
18.7.2	Jini.....	845
18.7.3	JavaSpaces.....	847
18.7.4	Java Management Extensions (JMX)	849
18.8	Web Services.....	850
18.8.1	Microsoft's .NET Platform.....	852
18.8.2	Sun Microsystems and the Sun ONE Platform.....	853

Part 7 Security..... 866

19	Security	869
19.1	Introduction.....	872
	Operating Systems Thinking: Ethical Systems Design.....	872
19.2	Cryptography.....	873
19.2.1	Secret-Key Cryptography.....	875
19.2.2	Public-Key Cryptography.....	877
	Biographical Note: Rivest, Shamir and Adleman.....	880
19.3	Authentication.....	880
19.3.1	Basic Authentication.....	881
19.3.2	Biometrics and Smart Cards.....	882
19.3.3	Kerberos.....	884
19.3.4	Single Sign-On.....	885
19.4	Access Control.....	886
19.4.1	Access Rights and Protection Domains.....	886
19.4.2	Access Control Models and Policies.....	887
19.4.3	Access Control Mechanisms	888
19.5	Security Attacks	892
19.5.1	Cryptanalysis.....	892
19.5.2	Viruses and Worms	892
19.5.3	Denial-of-Service (DoS) Attacks.....	894
19.5.4	Software Exploitation.....	896
19.5.5	System Penetration.....	896
19.6	Attack Prevention and Security Solutions.....	897

19.6.1	Firewalls.....	897
19.6.2	Intrusion Detection Systems (IDSs).....	899
19.6.3	Antivirus Software.....	900
19.6.4	Security Patches	902
19.6.5	Secure File Systems.....	903
	Mini Case Study: OpenBSD.....	904
	Mini Case Study: Macintosh.....	905
19.6.6	Orange Book Security.....	906
'9.7	Secure Communication.....	907
'9.8	Key Agreement Protocols.....	908
19.8.1	Key Management.....	908
19.8.2	Digital Signatures	910
19.9	Public-Key Infrastructure, Certificates and Certificate Authorities.....	911
19.10	Secure Communication Protocols.....	913
19.10.1	Secure Sockets layer (SSL).....	913
19.10.2	Virtual Private Networks (VPNs) and IP Security (IPSec)	914
19.10.3	Wireless Security.....	915
19.11	Steganography.....	917
19.12	Proprietary and Open-Source Security.....	918
19.13	Case Study: UNIX Systems Security.....	919

Part 8 Operating System Case Studies..... 936

20	Case Study: Linux.....	939
20.1	Introduction.....	942
20.2	History.....	942
20.3	Linux Overview.....	945
20.3.1	Development and Community.....	945
20.3.2	Distributions.....	946
20.3.3	User Interface.....	947
20.3.4	Stamfords.....	947
20.4	Kernel Architecture.....	948
20.4.1	Hardware Platforms.....	950
	Mini Case Study: User-Mode Linux (UML).....	951
20.4.2	Loadable Kernel Modules.....	952

20.5 Process Management.....	952
20.5.1 Process and Thread Organization.....	953
20.5.2 Process Scheduling.....	955
20.6 Memory Management.....	961
20.6.1 Memory Organization.....	961
20.6.2 Physical Memory Allocation and Deallocation.....	966
20.6.3 Page Replacement.....	968
20.6.4 Swapping.....	970
20.7 File Systems.....	971
20.7.1 Virtual File System.....	972
20.7.2 Virtual File System Caches.....	975
20.7.3 Second Extended File System (ext2fs).....	977
20.7.4 Proc File System.....	982
20.8 Input/Output Management.....	983
20.8.1 Device Drivers.....	983
20.8.2 Character Device I/O.....	986
20.8.3 Block Device I/O.....	987
20.8.4 Network Device I/O.....	991
20.8.5 Unified Device Model.....	992
20.8.6 Interrupts.....	995
20.9 Kernel Synchronization.....	997
20.9.1 Spin locks.....	997
20.9.2 Reader/Writer locks.....	998
20.9.3 Seqlocks.....	998
20.9.4 Kernel Semaphores.....	999
20.10 Interprocess Communication.....	1000
20.10.1 Signals.....	1001
20.10.2 Pipes.....	1002
20.10.3 Sockets.....	1003
20.10.4 Message Queues.....	1005
20.10.5 Shared Memory.....	1005
20.10.6 System V Semaphores.....	1007
20.11 Networking.....	1008
20.11.1 Packet Processing.....	1008
20.11.2 Netfilter Framework and Hooks.....	1010

20.12 Scalability.....	1010
20.12.1 Symmetric Multiprocessing (SMP).....	1011
20.12.2 Nonuniform Memory Access (NUMA).....	1012
20.12.3 Other Scalability Features.....	1013
20.12.4 Embedded Linux.....	1014
20.13 Security	1014
20.13.1 Authentication.....	1015
20.13.2 Access Control Methods.....	1015
20.13.3 Cryptography.....	1017
21 Case Study: Windows XP.....	1033
21.1 Introduction.....	1036
21.2 History.....	1036
Biographical Note: Bill Gates.....	1037
Biographical Note: David Cutler.....	1039
Mini Case Study: OS/2.....	1040
21.3 Design Goals.....	1041
21.4 System Architecture.....	1042
21.5 System Mamgement Mechanisms.....	1044
21.5.1 Registry.....	1044
21.5.2 Object Manager.....	1045
21.5.3 Interrupt Request Levels (IRQLs)	1046
21.5.4 Asynchronous Procedure Calls (APCs).....	1048
21.5.5 Deferred Procedure Calls (DPCs)	1049
21.5.6 System Threads.....	1051
21.6 Process and Thread Management.....	1051
21.6.1 Process and Thread Oranization.....	1052
21.6.2 Thread Scheduling.....	1055
21.6.3 Thread Synchronization.....	1059
21.7 Memory Management	1064
21.7.1 Memory Organization.....	1065
21.7.2 Memory Allocation.....	1067
21.7.3 Page Replacement.....	1071
21.8 File Systems Management.....	1073
21.8.1 File System Drivers.....	1074

21.8.2 NTFS.....	1074
21.9 Input/Output Management.....	1080
21.9.1 Device Drivers.....	1081
21.9.2 Input/Output Processing.....	1084
21.9.3 Interrupt Handling.....	1089
21.9.4 File Cache Management.....	1090
21.10 Interprocess Communication.....	1091
21.10.1 Pipes.....	1091
21.10.2 Mailslots.....	1093
21.10.3 Shared Memory.....	1094
21.10.4 Local and Remote Procedure Calls.....	1094
21.10.5 Component Object Model (COM).....	1097
21.10.6 Drag-and-Drop and Compound Documents.....	1098
21.11 Networking.....	1099
21.11.1 Network Input/Output.....	1099
21.11.2 Network Driver Architecture.....	1101
21.11.3 Network Protocols.....	1103
21.11.4 Network Services.....	1105
21.11.5 .NET.....	1105
21.12 Scalability.....	1106
21.12.1 Symmetric Multiprocessing (SMP).....	1106
21.12.2 Windows XP Embedded.....	1108
21.13 Security.....	1108
21.13.1 Authentication.....	1109
21.13.2 Authorization.....	1110
21.13.3 Internet connection Firewall.....	1111
21.13.3 Other Features.....	1112
Glossary.....	1138
Index.....	1188

Illustration

<i>Part 1 Introduction to Hardware, Software and Operating System.....</i>	<i>1</i>
--	----------

1 Introduction to Operating Systems

<i>1.1 Interaction between applications and the operating system.</i>	<i>25</i>
<i>1.2 Schematic of a virtual machine.....</i>	<i>28</i>
<i>1.3 Monolithic operating system kernel architecture.....</i>	<i>34</i>
<i>1.4 Layers of THE operating system.....</i>	<i>36</i>
<i>1.5 Microkernel operating system architecture.....</i>	<i>37</i>
<i>1.6 Client/server networked operating system model.....</i>	<i>38</i>

2 Hardware and Software Concepts

<i>2.1 Transistor or count plotted against time for Intel processors.</i>	<i>58</i>
<i>2.2 Process or components.</i>	<i>61</i>
<i>2.3 Memory hierarchy.....</i>	<i>64</i>
<i>2.4 Direct memory access (DMA).</i>	<i>69</i>
<i>2.5 Peripheral devices.</i>	<i>71</i>
<i>2.6 Bootstrapping.....</i>	<i>77</i>
<i>2.7 Application programming interface (API).</i>	<i>85</i>
<i>2.8 Compiler phases.</i>	<i>86</i>
<i>2.9 Object module.....</i>	<i>87</i>
<i>2.10 Linking process.</i>	<i>88</i>
<i>2.11 Symbol resolution.....</i>	<i>89</i>
<i>2.12 Loading.....</i>	<i>92</i>
<i>2.13 Compiling, linking and loading.....</i>	<i>92</i>

<i>Part 2 Processes and Threads.....</i>	<i>106</i>
--	------------

3 Process Concepts

<i>3.1 Process state transitions.....</i>	<i>115</i>
<i>3.2 Process table and process control blocks.....</i>	<i>117</i>
<i>3.3 Process creation hierarchy.....</i>	<i>119</i>

3.4 Process hierarchy in Linux.....	120
3.5 Process state transitions with suspend and resume.....	121
3.6 Context switch.....	122
3.7 Handling interrupts.....	126
3.8 Common interrupt types recognized in the Intel IA-32 architecture.....	128
3.9 Intel IA-32 exception classes.	128
3.10 UNIX system calls.	135
3.11 Code for Exercise 3.4.	139

4 Thread Concepts

4.1 Thread relationship to processes.....	149
4.2 Thread life cycle.....	153
4.3 User-level threads.....	156
4.4 Kernel-level threads.....	157
4.5 Hybrid threading model	160
4.6 Signal masking.....	164
4.7 Linux task state-transition diagram.	169
4.8 Windows XP thread state-transition diagram.	171
4.9 Java-threads being created, starting, sleeping and printing.....	174

5 Asynchronous Concurrent Execution

5.1 Buffer interface used in producer/consumer examples.	190
5.2 Producer class represents the producer thread in a producer/consumer relationship.....	191
5.3 Consumer class represents the consumer thread in a producer/consumer relationship.	192
5.4 UnsynchronizedBuffer class maintains the shared integer that is accessed by a producer thread and a consumer thread via methods set and get.....	194
5.5 SharedBuffer class enables threads to modify a shared object without synchronization.	195
5.6 Mutual exclusion implementation—version 1.....	201
5.7 Mutual exclusion implementation—version 2.	203
5.8 Mutual exclusion implementation—version 3.	205
5.9 Mutual exclusion implementation—version 4.	206
5.10 Dekker's Algorithm for mutual exclusion.	208
5.11 Peterson's Algorithm for mutual exclusion.....	212
5.12 Lamport's Bakery Algorithm.....	217
5.13 testAndSet instruction for mutual exclusion.....	223
5.14 swap instruction for mutual exclusion.....	225
5.15 Mutual exclusion with semaphores.....	228
5.16 Producer/consumer relationship implemented with semaphores.	230
5.17 Code for exercise 5.5.....	237
5.18 New mutual exclusion primitives for Exercise 5.16(a).....	238
5.19 Code for Exercise 5.16(b).....	238
5.20 Algorithm for Exercise 5.28.....	239

6 Concurrent Programming

6.1 Simple resource allocation with a monitor in pseudocode	253
6.2 Monitor pseudocode implementation of a circular buffer.....	254
6.3 Monitor pseudocode for solving the readers and writers problem.	258
6.4 SynchronizedBuffer synchronizes access to a shared integer.....	262
6.5 Threads modifying a shared object with synchronization.....	267
6.6 SynchronizedBuffer controls access to a shared array of integers.	271
6.7 CircularBufferTest instantiates producer and consumer threads.....	276

7 Deadlock and Indefinite Postponement

7.1 Traffic deadlock example.....	292
7.2 Resource deadlock example: This system is deadlocked because each process holds a resource being requested by the other process and neither process is willing to release the resource it holds.	294
7.3 Dining philosopher behavior.....	296
7.4 Implementation of method eat	297
7.5 Havender's linear ordering of resources for preventing deadlock.....	306
7.6 Safe state.	309
7.7 Unsafe state.....	310
7.8 Safe-state-to-unsafe-state transition.....	310
7.9 State description of three processes.	311
7.10 Resource allocation and request graphs.....	314
7.11 Graph reductions determining that no deadlock exists.....	315
7.12 Typical philosopher behavior for Exercise 7.9.....	323
7.13 Philosopher behavior for Exercise 7.9(a).....	323
7.14 Philosopher behavior for Exercise 7.9(b).....	324
7.15 Philosopher behavior for Exercise 7.9(c).....	324
7.16 Philosopher behavior for Exercise 7.9(d).....	325
7.17 Resource description for State A	326
7.18 Resource description for State B.	327
7.19 System state describing current loan, maximum need and current claim.....	327
7.20 System state describing total number of resources and available resources.....	327
7.21 Example of a system in an unsafe state.	328

8 Processor Scheduling

8.1 Scheduling levels.....	336
8.2 First-in-first-out scheduling.	344
8.3 Round-robin scheduling.....	345
8.4 Multilevel feedback queues.	351
8.5 Standard UNIX process scheduler. The scheduler grants the processor to users, each of whom may have many processes. (Property of AT&T Archives. Reprinted with permission of AT&T.).....	355

8.6 Fair share sheduler. The fair share scheduler divides system resource capacity into portions, which are then allocated by process schedulers assigned to various fair share groups. (Properly of AT&T Archives. Reprinted with permission of AT&T)	356
8.7 Java thread priority scheduling.....	362

Part 3 Physical and Virtual Memory 374

9 Real Memory Organization and Management

9.1 Microsoft Windows operating system memory requirements.	381
9.2 Hierarchical memory organization.....	383
9.3 Single-user contiguous memory allocation.....	385
9.4 Overlay structure.	386
9.5 Memory protection with single-user contiguous memory allocation.	388
9.6 Processor utilization on a single-user system. [Note: In many single user jobs, I/O waits are much longer relative to processor utilization periods indicated in this diagram.].....	390
9.7 Fixed-partition multiprogramming with absolute translation and loading.....	391
9.8 Memory waste under fixed-partition multiprogramming with absolute translation and loading.	392
9.9 Fixed-partition multiprogramming with relocatable translation and loading.....	392
9.10 Memory protection in contiguous-allocation multiprogramming systems.....	393
9.11 Internal fragmentation in a fixed-partition multiprogramming system.	395
9.12 Initial partition assignments in variable-partition programming.	396
9.13 Memory "holes" in variable-partition multiprogramming.	397
9.14 Coalescing memory "holes" in variable-partition multiprogramming.	398
9.15 Memory compaction in variable-partition multiprogramming.....	398
9.16 First-fit, best-fit and worst-fit memory placement strategies.....	400
9.17 Multiprogramming in a swapping system in which only a single process at a time is in main memory.	402

10 Virtual Memory Organization

10.1 Evolution of memory organizations.....	416
10.2 Two-level storage	419
10.3 Pieces of address spaces exist in memory and in secondary storage.....	420
10.4 Mapping virtual addresses to real addresses.	421
10.5 Artificial contiguity.....	422
10.6 Virtual address format in a block mapping system.....	422
10.7 Virtual address translation with block mapping.	423
10.8 Virtual address format in a pure paging system.....	424
10.9 Main memory divided into page frames.....	425
10.10 Correspondence between virtual memory addresses and physical memory addresses in a pure paging system.....	426

10.11 Page table entry.	427
10.12 Paging address translation by direct mapping.	428
10.13 Paging address translation with pure associative mapping.....	430
10.14 Paging address translation with combined associative/direct mapping.	431
10.15 Multilevel page address translation.....	434
10.16 Page address translation using inverted page tables.....	437
10.17 Inverted page table using a hash anchor table.....	438
10.18 Sharing in a pure paging system.	440
10.19 Noncontiguous memory allocation in a real memory segmentation system.....	443
10.20 Virtual address format in a pure segmentation system.	443
10.21 Virtual address translation in a pure segmentation system.....	444
10.22 Segment map table entry.....	445
10.23 Sharing in a pure segmentation system.....	447
10.24 Memory protection with keys in noncontiguous memory allocation multiprogramming systems.	448
10.25 Access control types.	449
10.26 Combining read, write and execute access to yield useful access control modes.....	449
10.27 Segment map table entry with protection bits.....	450
10.28 Virtual address format in a segmentation/paging system.	451
10.29 Virtual address translation with combined associative/direct mapping in a segmentation/paging system.....	452
10.30 Table structure for a segmentation/paging system.....	454
10.31 Two processes sharing a segment in a segmentation/paging system.	456
10.32 Virtual machine multiprogramming.....	463

11 Virtual Memory Management

11.1 Space-time product under demand paging.....	488
11.2 First-in-first-out (FIFO) page replacement.	487
11.3 FIFO anomaly-page faults can increase with page frame allocation.	489
11.4 Least-recently-used (LRU) page replacement strategy.....	490
11.5 Page types under NUR.	492
11.6 Far page replacement strategy access graph.	495
11.7 Storage reference pattern exhibiting locality. (Reprinted by permission from IBM Systems Journal. ©1971 by International Business Machines Corporation.).....	496
11.8 Dependency of page fault rate on amount of memory for a process's pages.....	497
11.9 Definition of a process's working set of pages.....	498
11.10 Working set size as a function of window size.....	499
11.11 Main memory allocation under working set memory management.....	499
11.12 Internal fragmentation in a paged and segmented system.	503
11.13 Page sizes in various process or architectures.....	504
11.14 Percentage of a process's pages referenced with time.....	505
11.15 Dependency of interfault time on the number of page frames allocated to a process.....	506
11.16 Linux page replacement overview.	509

Part 4 Secondary Storage, Files and Databases..... 522

12 Disk Performance Optimization

12.1 Schematic side view of a moving-head disk.	529
12.2 Hard disk track-to-track seek times and latency times.....	530
12.3 Schematic top view of a disk surface.	531
12.4 Components of a disk access.....	532
12.5 Seek request pattern.	534
12.6 Seek pattern under the FCFS strategy.....	535
12.7 Seek pattern under the SSTF strategy.....	537
12.8 Seek pattern under the SCAN strategy.	538
12.9 Seek pattern under the C-SCAN strategy.	539
12.10 Seek pattern under the FSCAN strategy.....	540
12.11 Seek pattern under the N-Step SCAN strategy (n = 3).	540
12.12 Seek pattern under the LOOK strategy.	541
12.13 Seek optimization strategies summary.....	542
12.14 SLTF scheduling. The requests will be serviced in the indicated order regardless of the order in which they arrived.	543
12.15 SPTF (a) and SATF(b) disk scheduling examples.	544
12.16 Strips and stripe created from a single file in RAID systems.	553
12.17 RAID level 0 (striping).....	556
12.18 RAID level 1 (mirroring).	557
12.19 RAID level 2 (bit-level ECC parity).....	559
12.20 RAID level 3 (bit-level, single parity disk).....	561
12.21 RAID level 4(block-level parity).....	563
12.22 RAID level 5 (block-level distributed parity).	564
12.23 Comparison of RAID levels 0-5.....	565

13 File and Database System:

13.1 Directory file contents example.....	591
13.2 Two-level hierarchical file system	592
13.3 Example hierarchical file system contents.	593
13.4 Links in a file system.....	594
13.5 Mounting a file system.....	597
13.6 Noncontiguous file allocation using a linked list.....	601
13.7 Tabular noncontiguous file allocation.....	603
13.8 Index block chaining.	606
13.9 Inode structure.	607
13.10 Free space management using a free list.....	608
13.11 Free space management using a bitmap.....	609
13.12 Access control matrix.....	611
13.13 Log-structured file system.....	618

13.14 Relation in a relational database.....	622
13.15 Relation formed by projection.....	623
13.16 SQL query.....	623

Part 5 Performance, Processors and Multiprocessor Management..... 638

14 Performance and Processor Design

14.1 Summary of performance evaluation techniques.....	657
14.2 The 10 most frequently executed instructions on IBM's system/370 architecture. (Courtesy of International Business Machines Corporation.).....	663
14.3 RISC and CISC comparison.....	665
14.4 Instruction execution in an (a) EPIC processor and (b) post-RISC processor.	669

15 Multiprocessor Management

15.1 Shared bus multiprocessor organization.	691
15.2 4-connected 2-D mesh network.....	693
15.3 Crossbar-switch matrix multiprocessor organization.	693
15.4 3- and 4-dimensional hypercubes.....	694
15.5 Multistage baseline network.	695
15.6 Tightly coupled system.....	697
15.7 Loosely coupled system.....	697
15.8 Master/Slave multiprocessing.	698
15.9 UMA multiprocessor.	702
15.10 NUMA multiprocessor.....	703
15.11 COMA multiprocessor.	705
15.12 NORMA multiprocessor.	706
15.13 Coscheduling (undivided version).....	717
15.14 Dynamic partitioning.....	718
15.15 Process migration.	720
15.16 Static load balancing using graph.....	725
15.17 Processor load diffusion.	727

Part 6 Networking and Distributed Computing. 746

16 Introduction to Networking

16.1 Network topologies.....	752
16.2 FTP commands.....	758
16.3 Sending a message via the Token Ring protocol.	767
16.4 Three-tier client/server model	769

17 Introduction to Distributed Systems

17.1 RPC communication model.....	790
17.2 Wound-wait strategy	799
17.3 Wait-die strategy.....	799
17.4 System without deadlocks.....	801
17.5 Deadlock is introduced to the system.....	802
17.6 System after deadlock has been eliminated.	802

18 Distributed System and Web Services

18.1 NFS architecture.....	820
18.2 AFS structure.....	822
18.3 Coda volume structure.....	825
18.4 File consistency issues in mutually exclusive AVSGs.....	826
18.5 Sprite file system domains.	827
18.6 Reading and writing in the Sprite file system.....	829
18.7 Typical Beowulf cluster.....	834
18.8 High-availability cluster with shared storage.	834
18.9 High-availability cluster with local storage	835
18.10 Load-balancing cluster with 32 nodes.	835
18.11 Centralized P2P instant-messaging application.	837
18.12 Pure P2P instant-messaging application.	838
18.13 Common P2P applications.....	838
18.14 JXTA low-level protocols.	841
18.15 Jini architecture.....	846
18.16 Distributed image-processing application using JavaSpaces.....	848
18.17 JMX's three-level management architecture.....	850
18.18 P2P system with seven peers.	860

Part 7 Security..... 866

19 Security

19.1 Encrypting and decrypting a message using a secret key.	875
19.2 Distributing a session key with a key distribution center.	876
19.3 Encrypting and decrypting a message using public-key cryptography.....	877
19.4 Authentication with a public-key algorithm.....	879
19.5 Salting passwords (Base 64 encoding).	882
19.6 Access control matrix for a small group of subjects and objects.....	889
19.7 Access control list derived from the access control matrix.	890
19.8 Polymorphic virus.	901
19.9 Creating a digital envelope.....	909

Part 8 Operating System Case Studies..... 96

20 Cast Study: Linux

20.1 Linux architecture.....	949
20.2 Task state-transition diagram.....	954
20.3 Scheduler priority array.....	956
20.4 Priority values and their corresponding levels of interactivity.....	959
20.5 Page table organization.....	962
20.6 Kernel virtual address space mapping.	964
20.7 Physical memory zones on the IA-32 Intel architecture.....	965
20.8 free_area vector.....	967
20.9 Page-replacement system overview.	969
20.10 Relationship between the VFS, file systems and data.	972
20.11 Dentry organization for a particular /home directory.	974
20.12 VFS file and inode operations.....	975
20.13 Dentry and inode caches.	976
20.14 Ext2 inode contents.....	978
20.15 Block group.	979
20.16 Group descriptor.....	980
20.17 Directory structure.....	981
20.18 Sample contents of the /proc directory.	982
20.19 /proc/devices file contents.....	984
20.20 I/O interface layers.....	985
20.21 chrdevs vector.....	986
20.22 Block I/O subsystem layers.....	987
20.23 Unified device model organization.....	994
20.24 POSIX signals	1001
20.20 System V shared memory system calls.	1006
20.26 Path followed by network packets received by the networking subsystem.	1009
20.27 Netfilter hooks.....	1010
20.28 Loopback device providing an encrypted file system using the Cryptographic API	1017

21 Case Study: Windows XP

21.1 Windows XP system architecture.....	1043
21.2 Interrupt request levels (IRQLs).....	1048
21.3 Thread state-transition diagram.....	1056
21.4 Dispatcher objects in Windows XP.....	1061
21.5 Virtual address translation.....	1066
21.6 Page states.....	1066
21.7 Memory allocation stages.	1068
21.8 Page frame states.	1069

21.9 Page-replacement process..... 1072

21.10 Master File Table (MFT) entry for a sample file..... 1076

21.11 Directory contents are stored in B-trees..... 1077

21.12 Windows XP I/O support components 1081

21.13 I/O request packet (IRP). 1085

21.14 Major function code examples in Windows XP. 1085

21.15 Servicing an IRP. 1087

21.16 Flow of an LPC or RPC call..... 1095

21.17 Handling a network I/O request. 1100

21.18 Network driver architecture.1102

21.19 Authentication process in a Windows XP network 1110

Preface

Live in fragments no longer. Only connect.

Edward Morgan Forster

Welcome to the world of operating systems. This text is intended primarily for use in the one-semester and two-semester operating systems courses (as defined in the most recent ACM/IEEE curriculum) that universities offer to juniors, seniors and graduate students in computer science. Operating systems designers and systems programmers will find the text useful as a reference as well.

The text features extensive case studies on two of today's most important operating systems—Linux (94 pages) and Microsoft® Windows® XP (106 pages)—that represent two different operating system design paradigms—free, open-source development and licensed, corporate development, respectively. The Linux case study follows the development of the kernel through version 2.6. The Windows XP case study highlights the internals of the current version of the most widely used personal computer operating system. These case studies enable you to compare and contrast the design and implementation philosophies used in real-world operating systems.

Both Linux and Windows XP are massive, complex operating systems containing millions of lines of source code. We survey the major components of each operating system. The case studies present issues in personal computer, workstation, multiprocessor, distributed and embedded environments, including a detailed discussion on why Linux and other UNIX-like operating systems have become prominent in the open-source and open-systems philosophies of major corporations.

This preface introduces our teaching approach in *Operating Systems, 3/e* and the book's key content and design elements. We also discuss the ancillary support available with the text. The section entitled "Tour of the Book" overviews the rich coverage of operating systems this book provides.

Operating Systems, 3/e, was reviewed by a team of distinguished academics and industry professionals; their names and affiliations are listed in the Acknowledgements section.

As you read this book, if you have any questions, please send us an e-mail at [**deitel@deitel.com**](mailto:deitel@deitel.com); we will respond promptly. Please visit our Web site, [**www.deitel.com**](http://www.deitel.com), regularly and sign up for the *Deitel Buzz Online* e-mail newsletter at [**www.deitel.com/newsletter/subscribe.html**](http://www.deitel.com/newsletter/subscribe.html). We use the Web site and the newsletter to keep our readers current on all Deitel publications and services. The site [**www.deitel.com/books/os3e**](http://www.deitel.com/books/os3e) is dedicated to *Operating Systems, 3/e*. At that site, we will post various ancillary materials for students and instructors, including dependency charts, content updates, errata, suggested student research projects (term projects, directed studies and theses), sample syllabi and the bibliography of research materials from the prior edition of the book. Prentice Hall provides a Companion Website for the book at [**www.prenhall.com/deitel**](http://www.prenhall.com/deitel).

Book Design

Operating Systems, 3/e features a completely new, two-color design inspired by the work of Leonardo DaVinci. The two-color design is visually appealing and allows us to use color to improve the pedagogy of the book. For example, defining occurrences of key terms appear in **bold and color** so they stand out to you.

Operating Systems, 3/e Features

Operating Systems, 3/e includes extensive new content. We also revised and updated much of the material from the second edition. The focus on current technologies and issues in distributed computing makes this book unique from its competition. Several new sections have been added to address embedded, real-time and distributed systems. Some key features of this edition are:

- Conforms to all core requirements of the ACM/IEEE CC2001 Operating Systems course requirements.
- Discusses all the CC2001 elective operating systems topics except shell scripting.
- Provides an updated hardware introduction that includes leading-edge technologies and their impact on operating systems design.
- Presents process, thread, memory and disk management solutions that reflect the needs of current applications.
- Supplements the extensive coverage of general-purpose systems with concepts relevant to real-time, embedded and superscalar architectures.
- Highlights key evaluation techniques that enable effective comparative analyses of operating system components.
- Includes a richer treatment of networking concepts.
- Enhances the security treatment to include current trends in authentication mechanisms, security protocols, anti-virus research, access control methods and wireless security.

- Enhances the distributed computing coverage to acknowledge the tremendous influence of the Internet and the World Wide Web on computing and operating systems.
- Provides details of the ubiquitous Intel architecture.
- Provides many diagrams, tables, working code examples, pseudocode examples and algorithms.
- Includes a new chapter on threads.
- Pseudocode is presented in Java-like syntax to capitalize on C/C++/Java literacy—virtually all computer-science students know one or more of these languages.
- Provides multithreading treatments in both pseudocode and Java that demonstrate issues in concurrent programming—enabling instructors to cover the material the way they prefer. The Java treatment is new to this edition and is optional. Visit java.sun.com/j2se/downloads.html to obtain the latest version of Java. The download page contains a link to the installation instructions.
- Enhances the multiprocessor management treatment.
- Provides new sections on thread scheduling and real-time scheduling.
- Includes a RAID discussion.
- Provides a case study on UNIX processes.
- Includes up-to-the-minute sections on memory-management and disk-scheduling strategies.
- Covers the important topic of I/O systems in many chapters—most notably Chapters 2, 12, 13 and the case study chapters (20 and 21).
- Provides 730 self-review questions and answers (approximately two per section) for immediate feedback.
- Includes extensive research with citations listed in the Works Cited section at the end of each chapter.

Teaching Approach

This book is divided into eight parts, each containing a set of related chapters. The parts are:

1. Introduction to Hardware, Software and Operating Systems
2. Processes and Threads
3. Real and Virtual Memory
4. Secondary Storage, Files and Databases
5. Performance, Processors and Multiprocessor Management
6. Networking and Distributed Computing

7. Security
8. Case Studies

The book's pedagogic features are described below.

Quotations

Each chapter begins with quotations—some are humorous, some are philosophical and some offer interesting insights. Many readers have told us that they enjoy relating the quotations to the chapter material. You might appreciate some of the quotations more *after* reading the chapters.

Objectives

Next, objectives tell you what to expect and give you an opportunity, after reading the chapter, to determine whether you have met these objectives.

Outline

The chapter outline helps you approach the material in a top-down fashion, so you can anticipate the concepts that will be presented in each chapter and set a comfortable learning pace.

Sections and Self-Review Exercises

Each chapter contains small sections that address important operating systems concepts. Most sections end with two self-review exercises and answers. These exercises enable you to test your knowledge, get immediate feedback and gauge your understanding of the material. These exercises also help prepare you for the end-of-chapter exercises and for quizzes and exams. Some of the self-review exercises cannot be answered only from the material presented in their corresponding sections; these are additional teaching and learning opportunities.

Key Terms

The defining occurrence of each term appears in **bold and color**. In addition, all chapters include a Key Terms section containing the terms defined in the chapter and their definitions (1800+ key terms in the text). A cumulative alphabetized glossary appears at the end of the book. The Key Terms sections and the Glossary are wonderful pedagogic devices for review and reference.

Figures

The text contains over 300 charts, diagrams, examples and illustrations that support the concepts presented in the text.

Web Resources

Each chapter contains Web resources that direct you to sites where you can locate valuable additional research materials.

Summary

Detailed end-of-chapter summary sections help you review the key concepts presented in each chapter.

Exercises, Suggested Projects and Suggested Simulations

Each chapter includes many exercises that vary in difficulty from review of basic operating systems principles to complex reasoning and research projects (900+ exercises in the text). Many OS instructors like to assign term projects, so we have included Suggested Projects and Suggested Simulations sections at the end of each chapter's exercises.

Recommended Readings

Each chapter lists recommended books and articles, and provides a brief review of the most important sources for the chapter content so you can do additional research on your own.

Works Cited

This book required an extraordinary research effort. The entire book is thoroughly cited (2300+ citations). Each citation appears as a superscript in the text and corresponds to an entry in the Works Cited section at the end of the chapter. Many of these citations are to Web sites. Prior editions of this text used only standard book and literature citations; those books and papers were often difficult for readers to locate for further research. Now you can access these citations directly via the Web. Also, it is easy to use search engines to locate additional articles on subjects of interest. Many research journals are accessible online—some are free and others are available through personal or organizational memberships in professional societies. The Web is truly a research bonanza that leverages your learning experience.

Index

We provide a two-level index to help you locate any term or concept quickly. For example, each operating system mentioned in the text appears in the index both alphabetically and indented under the term "Operating Systems."

Feature Boxes

The second edition of *Operating Systems* included a full chapter on analytic modeling with queuing theory and Markov processes. This edition omits that material, recognizing that operating systems is for the most part not a mathematical field. Rather it has a basis in what we call "systems thinking"—operating systems is largely a field of empirical results. To illuminate these issues, we have included in this edition four types of feature boxes, presenting material to challenge, entertain and enrich you.

Biographical Notes

Eighteen Biographical Notes provide interesting details about people who have made significant contributions to the field of operating systems—*Edsger Dijkstra, Linus Torvalds, David Cutler, Ken Thompson, Dennis Ritchie, Doug Engelbart, Tim Berners-Lee, Richard Stallman, Gordon Moore, Fernando J. Corbato, Leslie Lamport, Per Brinch Hansen, Peter Denning, Seymour Cray, Bill Gates, Ronald Rivest, Adi Shamir and Leonard Adleman*. Please do not attach any particular significance to our choices; these are only a few among thousands of important contributors. Please let us know of other people you feel deserve mention.

Mini Case Studies

In addition to the detailed case studies on the Linux and Windows XP operating systems, 14 Mini Case Studies focus on other important operating systems of historical research or commercial interest. These Mini Case Studies include *Mach, CTSS and Multics, UNIX Systems, Real-Time Operating Systems, Atlas, IBM Mainframe Operating Systems, Early History of the VM Operating System, MS-DOS, Supercomputers, Symbian OS, OpenBSD, Macintosh, User-Mode Linux (UML) and OS/2*.

Anecdotes

One of the authors, HMD, accumulated many anecdotes over decades of teaching operating systems in academia and industry. We have included 16 of these anecdotes. Some are humorous; others are thought provoking—raising deep philosophical issues. Each is a (hopefully) pleasant diversion from the book's technical discussions and concludes with a *Lesson to operating systems designers*.

Operating Systems Thinking

Academics enjoy the luxury of being able to study what is interesting about operating systems, especially clever algorithms, data structures and occasionally, areas that lend themselves nicely to mathematical analysis. Industry professionals must build real systems that work and meet the demanding cost, performance and reliability requirements of customers. Both kinds of thinking are rich with interesting issues. There are considerable overlaps as well as significant differences in what academics and industry professionals think about. This book aims to present a balanced treatment of both the academic and industry sides of operating systems theory and practice.

What is "operating systems thinking?" Forty-three Operating Systems Thinking features explore that question. Indeed, some aspects of operating systems lend themselves to sophisticated mathematical analysis. But the author's (HMD's) extended experience in the computer industry—42 years, including working (as a very junior person) on major operating systems research and development efforts at IBM and MIT, writing two earlier editions of this book and teaching operating systems in academia and industry dozens of times—has shown that these systems are far too complex for a significant mathematical treatment at the undergraduate or early graduate level. Even at the advanced graduate level, except for narrow areas of interest, operating systems defy mathematical analysis.

If there is not a mathematical basis for evaluating aspects of operating systems, how can we think about them productively? The answer is what we call "systems thinking," and the text certainly covers this. However, the Operating Systems Thinking features are our effort to capture key concepts that are prevalent in operating systems design and implementation.

The Operating Systems Thinking features are: *Innovation; Relative Value of Human and Computer Resources; Performance; Keep It Simple (KIS); Architecture; Caching; Legacy Hardware and Software; Principle of Least Privilege; Protection; Heuristics; Customers Ultimately Want Applications; Data Structures in Operating Systems; Asynchronism vs. Synchronism; Concurrency; Parallelism; Standards Conformance; Scalability; Information Hiding; Waiting, Deadlock and Indefinite Postponement; Overhead; Predictability; Fairness; Intensity of Resource Management vs. Relative Resource Value; There Are No Upper Limits to Processing Power, Memory, Storage and Bandwidth; Change Is the Rule Rather Than the Exception; Spatial Resources and Fragmentation; Visualization; Empirical Results and Locality-Based Heuristics; Lazy Allocation; Computer Theory in Operating Systems; Space-Time Trade-offs; Saturation and Bottlenecks; Compression and Decompression; Redundancy; Fault Tolerance; Mission-Critical Systems; Encryption and Decryption; Security; Backup and Recovery; Murphy's Law and Robust Systems; Graceful Degradation; Data Replication and Coherency and Ethical Systems Design*

Case Studies

Chapters 20 and 21 cover in depth the Linux and Windows XP operating systems, respectively. These thorough case studies were carefully reviewed by key Linux and Windows XP operating systems developers. The outlines for each of these case studies mimic the text's table of contents. The case studies truly reinforce the text's key concepts—the text presents the principles; the case studies show how these principles are applied in building today's two most widely used operating systems. The Linux case study follows the development of the latest kernel release (v. 2.6) and contains 262 citations. The Windows XP case study reflects the latest Windows operating system features and contains 485 citations.

Tour of the Book

This section provides an overview of the eight parts and 21 chapters of *Operating Systems, 3/e*.

Part 1—Introduction to Hardware, Software and Operating Systems—includes two chapters that introduce the notion of operating systems, present a history of operating systems and lay the groundwork of hardware and software concepts the reader will use throughout the book.

Chapter 1—Introduction to Operating Systems—defines the term "operating system" and explains the need for such systems. The chapter provides a historical perspective on operating systems, tracing their development decade by decade

through the second half of the 20th century. The batch-processing systems of the 1950s are considered. We see the 1960s trend towards parallelism with the advent of multiprogramming—both batch multiprogramming and interactive timesharing systems. We follow the development of key operating systems including CTSS, Multics, CP/CMS and Unix. The chapter considers the kinds of thinking that operating systems designers did in an era when computing resources were far more expensive than people resources (today the reverse is true). We follow the 1970s evolution of computer networking, the Internet and the TCP/IP protocol suite, and see the beginnings of the personal computing revolution. Personal computing matures in the 1980s with the release of the IBM personal computer and the Apple Macintosh, the latter popularizing the graphical user interface (GUI). We see the beginnings of distributed computing and the development of the client/server model. In the 1990s, Internet usage literally explodes with the availability of the World Wide Web. Microsoft becomes the world's dominant software maker and releases its Windows NT operating system (the ancestor of today's Windows XP operating system—the focus of Chapter 21). Object technology becomes the dominant development paradigm with languages like C++ and Java becoming popular. The rapid rise of the open-source-software movement leads to the phenomenal success of the Linux operating system—the focus of Chapter 20. We discuss how operating systems provide a platform for applications development. Embedded systems are considered with an emphasis on how crucial it is that mission-critical and business-critical systems be extraordinarily reliable. We consider the core components of operating systems and key operating systems goals. (The book maintains a focus on performance issues in virtually every aspect of operating systems.) Operating system architectures are introduced, including monolithic architecture, layered architecture, micro-kernel architecture and networked and distributed operating systems.

Chapter 2—Hardware and Software Concepts—summarizes the hardware and software resources operating systems manage. The chapter covers how trends in hardware design—most notably phenomenal increases in processing power, memory capacity and communications bandwidth—have affected operating system design and vice versa. Hardware components including mainboards, processors, clocks, main memory, secondary storage devices, buses, direct memory access (DMA), peripheral devices and more are discussed. We present recent and emerging hardware technologies, and discuss hardware support for operating systems, including processor execution modes, privileged instructions, timers, clocks, bootstrapping and plug-and-play. Performance enhancement techniques like caching and buffering are considered. Software concepts are examined including compiling, linking, loading, machine languages, assembly languages, interpreters, compilers, high-level languages, structured programming, object-oriented programming and application programming interfaces (APIs). The chapter also explains firmware and middleware.

Part 2—Processes and Threads—includes six chapters that present the notions of processes, threads, process- and thread-state transitions, interrupts, con-

text switching, asynchronism, mutual exclusion, monitors, deadlock and indefinite postponement, and processor scheduling of processes and threads.

Chapter 3—Process Concepts—begins our discussion of operating system primitives by defining the fundamental notion of process. We consider the life cycle of a process as it transitions between process states. The representation of a process as a process-control block or process descriptor is discussed with an emphasis on the importance of data structures in operating systems. The chapter motivates the need for process structures in an operating system and describes operations that can be performed on them, such as suspend and resume. Multiprogramming considerations, including suspending process execution and context switching, are introduced. The chapter discusses interrupts—a key to the successful implementation of any multiprogrammed environment. We consider interrupt processing and interrupt classes, interprocess communication with signals and message passing. We conclude with a case study of UNIX processes.

Chapter 4—Thread Concepts—extends our discussion of process concepts to a smaller unit of concurrent program execution: the thread. The chapter defines what threads are and explains their relationship to processes. The life cycle of a thread and how threads transition among various thread states is discussed. We consider various threading architectural models, including user-level threads, kernel-level threads and combining user- and kernel-level threads. The chapter presents thread implementation considerations, including thread signal delivery and thread termination. We discuss the POSIX standard and its threading specification, Pthreads. The chapter concludes with a presentation of the Linux, Windows XP and Java threading implementations. The Java presentation is accompanied by a complete, working Java program with sample outputs. [Note: The Java source code for all of the Java programs in the book is available for download at www.deitel.com/books/os3e.]

Chapter 5—Asynchronous Concurrent Execution—discusses the issues of concurrency encountered in multiprogrammed systems. The chapter introduces the problem of mutual exclusion and how threads must manage access to shared resources. A feature of the chapter is the Java Multithreading Case Study: A Producer/Consumer Relationship in Java—which uses a complete, working Java program and several sample outputs to clearly illustrate what happens when concurrent threads access shared data without synchronization. This example clearly shows that sometimes such a concurrent program will work fine and sometimes it will produce erroneous results. We show how to solve this problem with a multithreaded Java program in Chapter 6. The concept of the critical section in program code is introduced. Several software mechanisms that protect access to critical sections are presented as solutions to the mutual exclusion problem; these include Dekker's Algorithm, Peterson's Algorithm and *N*-thread mutual exclusion with Lamport's Bakery Algorithm. The chapter also discusses hardware mechanisms that facilitate the implementation of mutual exclusion algorithms; these include disabling interrupts, the test-and-set instruction and the swap instruction. Finally, semaphores are presented as a high-level mechanism for

implementing mutual exclusion and thread synchronization; both binary semaphores and counting semaphores are considered.

Chapter 6—Concurrent Programming—explains the notion of monitors (a high-level mutual exclusion construct) then proceeds to solve several classic problems in concurrent programming, using first pseudocode monitors then complete working Java programs with sample outputs. The monitor solutions are provided in a C/C++/Java-like pseudocode syntax. We explain how monitors enforce information hiding and discuss how monitor condition variables differ from "conventional" variables. The chapter illustrates how a simple monitor may be used to control access to a resource that requires exclusive use. We then discuss two classic problems in concurrent programming—the circular buffer and the readers-and-writers problem; we implement solutions to each of these with pseudocode monitors. Java monitors are explained and the differences between Java monitors and the ones described by the classic literature are discussed. The chapter continues our optional Java Multithreading Case Study by implementing a producer/consumer relationship and a circular buffer in Java. The student can use this latter program as a basis for implementing a solution to the readers-and-writers problem in Java.

Chapter 7—Deadlock and Indefinite Postponement—introduces two potentially disastrous consequences of waiting: deadlock and indefinite postponement. The key concern is that systems that manage waiting entities must be carefully designed to avoid these problems. Several examples of deadlock are presented, including a traffic deadlock, the classic one-lane bridge, a simple resource deadlock, deadlock in spooling systems and deadlock in Dijkstra's charming Dining Philosophers problem. Key resource concepts are considered, including preemptibility, sharing, reentrancy and serial reusability. The chapter formally defines deadlock and discusses deadlock prevention, avoidance, detection and recovery (almost invariably painful). The four necessary conditions for deadlock are explained, namely the "mutual-exclusion," "wait-for," "no-preemption" and "circular-wait" conditions. We examine Havender's methods for preventing deadlock by denying, individually, any of the last three conditions. Deadlock avoidance enables more flexible resource allocation than deadlock prevention. The chapter explains deadlock avoidance with Dijkstra's Banker's Algorithm, showing examples of a safe state, an unsafe state and a safe-state-to-unsafe-state transition. Weaknesses in the Banker's Algorithm are discussed. We explain deadlock detection with the resource-allocation graph reduction technique. The chapter concludes with a discussion of deadlock strategies in current and future systems.

Chapter 8—Processor Scheduling—discusses concepts and algorithms related to allocating processor time to processes and threads. Scheduling levels, objectives and criteria are considered. Preemptive and nonpreemptive scheduling approaches are compared. We explain how to carefully set priorities and quanta (finite-sized allocations of processor time) in scheduling algorithms. Several classic and current scheduling algorithms are considered, including first-in-first-out (FIFO), round-robin (RR), shortest-process-first (SPF), highest-response-ratio-next (HRRN),

shortest-remaining-time (SRT), multilevel feedback queues and fair-share scheduling. Each algorithm is evaluated using metrics such as throughput, average response time and the variance of response times. We discuss Java thread scheduling, soft real-time scheduling, hard real-time scheduling and deadline scheduling.

Part 3—Real and Virtual Memory—includes three chapters that discuss memory organization and memory management in real and virtual memory systems.

Chapter 9—Real Memory Management and Organization—presents a historical discussion of how real memory operating systems have organized and managed physical memory resources. The schemes have gone from the simple to the complex, ultimately seeking optimal usage of the relatively precious main memory resource. We review the memory hierarchy consisting of cache(s), primary memory and secondary storage. Then, three types of memory management strategies are discussed, namely fetch, placement and replacement. We present contiguous and noncontiguous memory allocation schemes. Single-user contiguous memory allocation is considered with a discussion of overlays, protection and single-stream batch processing. We trace the evolution of multiprogramming memory organizations from fixed-partition multiprogramming to variable-partition multiprogramming, considering issues including internal and external memory fragmentation, and presenting memory compaction and coalescing as means of reducing fragmentation. The chapter discusses the first-fit, best-fit and worst-fit memory placement strategies, and concludes with a discussion of multiprogramming with memory swapping.

Chapter 10—Virtual Memory Organization—describes fundamental virtual memory concepts and the hardware capabilities that support virtual memory. The chapter motivates the need for virtual memory and describes typical implementations. The key approaches to virtual memory organization—paging and segmentation—are explained and their relative merits are analyzed. We discuss paging systems, focusing on paging address translation by direct mapping, paging address translation by associative mapping, paging address translation with direct/associative mapping, multilevel page tables, inverted page tables and sharing in paging systems. The chapter investigates segmentation systems, focusing on segmentation address translation by direct mapping, sharing in a segmentation system, protection and access control in segmentation systems. We also examine hybridized segmentation/paging systems, considering dynamic address translation, sharing and protection in these systems. The chapter concludes by examining the popular IA-32 Intel architecture virtual-memory implementation.

Chapter 11—Virtual Memory Management—continues the discussion of virtual memory by analyzing how operating systems attempt to optimize virtual memory performance. Because paging systems have become dominant, we focus on page management in detail, most notably on page-replacement strategies. The chapter considers one of the most important empirical results in the field of operating systems, namely the locality phenomenon, and we consider the results from both the temporal and spatial perspectives. We discuss when pages should be brought into memory, examining both demand paging and anticipatory paging.

When available memory becomes scarce, incoming pages must replace pages already in memory—an operating system's page-replacement strategy can have an enormous impact on performance. Many page-replacement strategies are examined, including random, first-in-first-out (FIFO and Belady's Anomaly), least-recently-used (LRU), least-frequently-used (LFU), not-used-recently (NUR), second-chance, clock and segmented queue (SEGQ), far page and page-fault-frequency (PFF). Denning's classic working set model of program behavior is considered as well as how various page-replacement strategies attempt to achieve its goals. The chapter discusses the possibility of voluntary page release. We carefully examine the arguments for small pages and large pages, and how programs behave under paging. The chapter discusses Linux page replacement and concludes with a discussion of global vs. local page-replacement strategies.

Part 4—Secondary Storage, Files and Databases—includes two chapters that present techniques which operating systems employ to manage data on secondary storage. Hard-disk performance optimization, and file and database systems are discussed. Our treatment of I/O systems is distributed throughout the book, most notably in Chapters 2, 12 and 13 and in the case studies on Linux and Windows XP (Chapters 20 and 21, respectively).

Chapter 12—Disk Performance Optimization—focuses on the characteristics of moving-head disk storage and how the operating system can optimize its performance. The chapter discusses the evolution of secondary-storage devices and examines the characteristics of moving-head disk storage. During a half century of explosive technological development, moving-head disk storage devices continue to endure. We define why disk scheduling is necessary and demonstrate why it is needed to achieve high-performance from moving-head disk devices. The evolution of disk-scheduling strategies is presented, including seek-optimization strategies, including first-come-first-served (FCFS), shortest-seek-time-first (SSTF), SCAN, C-SCAN, FSCAN, N-Step SCAN, LOOK, C-LOOK and VSCAN (introduced in the exercises), and rotational-optimization strategies, including SLTF, SPTF and SATE. Other popular disk-system performance techniques are discussed, including caching, buffering, defragmentation, data compression and blocking. One of the most important additions to this third edition is the extensive treatment of RAID systems in this chapter. RAID (Redundant Arrays of Independent Disks) is a set of technologies that enable disk systems to achieve higher performance and fault tolerance. The chapter presents various key RAID "levels," including level 0 (striping), level 1 (mirroring), level 2 (bit-level Hamming ECC parity), level 3 (bit-level XOR ECC parity), level 4 (block-level XOR ECC parity) and level 5 (block-level distributed XOR ECC parity).

Chapter 13—File and Database Systems—discusses how operating systems organize and manage collections of data called files and databases. We explain key concepts, including the data hierarchy, files, file systems, directories, links, metadata and mounting. The chapter presents various file organizations, including sequential, direct, index sequential and partitioned. File allocation techniques are examined, including contiguous file allocation, linked-list noncontiguous file allocation, tabu-

lar noncontiguous file allocation and indexed noncontiguous file allocation. We explain file access control by user classes and access control matrices. Data access techniques are discussed in the context of basic access methods, queued access methods, anticipatory buffering and memory-mapped files. We explain techniques for ensuring data integrity, including protection, backup, recovery, logging, atomic transactions, rollback, commitment, checkpointing, shadow paging and using log-structured file systems. The chapter considers file servers and distributed systems and points the reader to the extensive treatment of these topics and others in Chapters 16-18. The chapter also introduces database systems, and analyzes their advantages, data access, the relational database model and operating system services that support database systems.

Part 5—Performance, Processors and Multiprocessor Management—includes two chapters that discuss performance monitoring, measurement and evaluation techniques, and focus on the extraordinary performance that can be achieved with systems that employ multiple processors.

Chapter 14—Performance and Processor Design—focuses on one of operating systems designers' most preeminent goals—system performance—and discusses the important role of specific processor types in achieving that goal. The chapter surveys performance measures, considering issues including absolute performance measures, ease of use, turnaround time, response time, system reaction time, variance in response times, throughput, workload, capacity and utilization. We discuss performance evaluation techniques, including tracing and profiling, timings, microbenchmarks, application-specific evaluation, analytic models, benchmarks, synthetic programs, simulation and performance monitoring. Bottlenecks and saturation are considered. We demonstrate how systems dynamically adjust to positive and negative feedback. A system's performance depends heavily on the performance of its processor(s), which is, in turn, heavily influenced by its instruction set architecture. The chapter examines key architectures, including complex instruction set computing (CISC), reduced instruction set computing (RISC) and various kinds of post-RISC processors. The chapter concludes with a discussion of explicitly parallel instruction computing (EPIC).

Chapter 15—Multiprocessor Management—provides an in-depth introduction to the hardware and software aspects of multiprocessing. One way to build ever more powerful computing systems is to employ multiple processors and possibly massive numbers of them. The discussion begins with a Mini Case Study on supercomputers followed by a Biographical Note on Seymour Cray, the father of supercomputing. We investigate multiprocessor architecture, considering issues including the classification of sequential and parallel architectures, processor interconnection schemes and loosely coupled vs. tightly coupled systems. Multiprocessor operating system organizations are examined, including master/slave, separate kernels and symmetrical organization. We explain memory access architectures, including uniform memory access (UMA), nonuniform memory access (NUMA), cache-only memory architecture (COMA) and no remote memory access (NORMA).

The chapter discusses multiprocessor memory sharing, considering issues of cache coherence, page replication and migration and shared virtual memory. We continue the discussion on processor scheduling which we began in Chapter 8, presenting job-blind multiprocessor scheduling and job-aware multiprocessor scheduling. Process migration is considered and issues of flow of process migration and process migration strategies are examined. We discuss multiprocessor load balancing, examining both static and dynamic load balancing strategies. The chapter explains how to enforce multiprocessor mutual exclusion with spin locks, sleep/wakeup locks and read/write locks.

Part 6—Networking and Distributed Computing—includes three chapters that present networks, networked systems and distributed systems.

Chapter 16—Introduction to Networking—introduces computer networking to lay the foundation for the following two chapters on distributed computing. The chapter discusses network topologies, including buses, rings, meshes, fully connected meshes, stars and trees and we explain the unique challenges posed by wireless networks. We consider local-area networks (LANs) and wide-area networks (WANs), and present a major treatment of the TCP/IP protocol stack. Application layer protocols including the Hypertext Transfer protocol (HTTP) and the File Transfer Protocol (FTP) are examined. We explain transport layer protocols including the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP). At the network layer the chapter explores the Internet Protocol (IP) and its latest version—Internet Protocol version 6 (IPv6). The chapter discusses link layer protocols, including Ethernet, Token Ring, Fiber Distributed Data Interface (FDDI) and IEEE 802.11 (wireless). The chapter concludes with a discussion of the client/server model and n -tier systems.

Chapter 17—Introduction to Distributed Systems—introduces distributed operating systems and discusses the attributes of distributed systems, including performance, scalability, connectivity, security, reliability, fault tolerance and transparency. We compare and contrast network operating systems and distributed operating systems. Communication in distributed systems and the crucial role played by "middleware" technologies are discussed, including remote procedure call (RPC), Remote Method Invocation (RMI), CORBA (Common Object Request Broker Architecture), and Microsoft's DCOM (Distributed Component Object Model). The chapter considers process migration, synchronization and mutual exclusion in distributed systems, examining mutual exclusion without shared memory, and Agrawala and Ricart's distributed mutual exclusion algorithm. We discuss deadlock in distributed systems, focusing on deadlock prevention and deadlock detection. The chapter concludes with case studies on the Sprite and Amoeba distributed operating systems.

Chapter 18—Distributed Systems and Web Services—continues the study of distributed systems, focusing on distributed file systems, clustering, peer-to-peer computing, grid computing, Java distributed computing and Web services. The chapter begins by considering some characteristics and concerns of distributed systems,

including transparency, scalability, security, fault tolerance and consistency and we present case studies on key distributed file systems, including the Network File System (NFS), Andrew File System (AFS), Coda File System and Sprite File System. We discuss clustering, considering high-performance clusters, high-availability clusters and load-balancing clusters; we investigate various examples of clusters including Linux-based Beowulf clusters Windows-based clusters. Peer-to-peer (P2P) distributed computing is explored considering the relationship between P2P and client/server applications, centralized vs. decentralized P2P applications, peer discovery and searching. We examine Sun Microsystems' Project JXTA and we consider how JXTA creates a framework for building P2P applications. The chapter considers grid computing and how it makes possible solutions to problems that require a truly massive amount of computation by using available unused processing power on personal and business computers worldwide. We discuss key Java distributed computing technologies, including Java servlets and JavaServer Pages (JSP), Jini, JavaSpaces and Java Management Extensions (JMX). The chapter concludes by surveying the exciting new technology of Web services and exploring two key Web services platforms—Microsoft's .NET Platform and Sun Microsystems' Sun ONE platform.

Part 7—Security—includes one chapter.

Chapter 19—Security—presents a general introduction to computer and network security techniques that operating systems can use to provide secure computing environments. The chapter discusses secret-key and public-key cryptography and the popular RSA and PGP (Pretty Good Privacy) public key schemes. Authentication is considered: password protection, password salting, biometrics, smart cards. Kerberos and single sign-on. We discuss access control, exploring issues of access rights, protection domains, access control models, access control policies and access control mechanisms, including access control matrices, access control lists and capability lists. The chapter presents the great variety of security attacks that have been attempted, including cryptanalysis, viruses, worms, denial-of-service (DoS) attacks, software exploitation and system penetration. We survey attack prevention and security solutions, including firewalls and intrusion detection systems (IDSs), antivirus software, security patches and secure file systems. We discuss the federal government's Orange Book Security classification system and present a Mini Case Study on OpenBSD, arguably the most secure operating system available. The chapter explores secure communication and consider the requirements for a successful, secure transaction—privacy, integrity, authentication, authorization and nonrepudiation. Key agreement protocols—the process by which two parties exchange secret keys over an unsecure medium—are discussed. We explain digital signatures—a technology that is enormously crucial to the future of electronic commerce—and their implementation. The chapter presents an in-depth treatment of public-key infrastructure, including digital certificates and certificate authorities. Various secure communication protocols are considered including Secure Sockets Layer (SSL), Virtual Private Networks (VPNs), IP Security (IPSec) and wireless security. We discuss the intriguing technology of steganography—the practice of

hiding information within other information. This can hide secret messages in publicly transmitted messages; it can also protect intellectual property rights, with digital watermarks for example. The chapter compares and contrasts proprietary and open-source security solutions and concludes with a case study on UNIX Security.

Part 8—Operating Systems Case Studies—includes two chapters that provide in-depth case studies on the Linux 2.6 kernel and the Microsoft Windows XP operating system. The case studies follow the book's outline to make it convenient for the student to comprehend topics discussed earlier in the book.

Chapter 20—Case Study: Linux—provides an in-depth study of Linux 2.6. This extensive (94 pages) case study was written and updated throughout the development of the release (by following the version 2.5 development kernel and having the material carefully reviewed by key Linux kernel developers). The chapter discusses the history, community and software distributions that have created the most popular open-source operating system in the world. The chapter examines the core components of the Linux operating system, with particular attention to their implementation in the context of concepts studied in previous chapters. Kernel architecture, process management (processes, threads, scheduling), memory organization and management, file systems (virtual file system, virtual file system caches, ext2fs and proc fs), I/O management (device drivers, character device I/O, block device I/O, network device I/O, unified device model, interrupts), synchronization (spin locks, reader/writer locks, seqlocks and kernel semaphores), IPC (signals, pipes, sockets, message queues, shared memory and System V semaphores), networking (packet processing, netfilter framework and hooks), scalability (symmetric multiprocessing, nonuniform memory access, other scalability features, embedded Linux) and security are all analyzed and explained.

Chapter 21—Case Study: Windows XP—complements the Linux case study by examining the internals of the most popular commercial operating system—Windows XP. This case study (106 pages) examines the core components of the Windows XP operating system and how they interact to provide services to users. We discuss the history of Windows operating systems, including Biographical notes on Bill Gates and David Cutler. The chapter presents the design goals of Windows XP and overviews its system architecture, considering topics including the Hardware Abstraction Layer (HAL), the microkernel, the executive, the environment subsystems, dynamic link libraries (DLLs) and system services. We discuss system management mechanisms, including the registry, Object Manager, interrupt request levels (IRQLs), asynchronous procedure calls (APCs), deferred procedure calls (DPCs) and system threads. We examine process and thread organization, considering control blocks, thread local storage (TLS), creating and terminating processes, jobs, fibers and thread pools. Thread scheduling is explained, and thread states, the thread scheduling algorithm, determining thread priorities and multiprocessor scheduling are considered. We investigate thread synchronization, examining dispatcher objects, event objects, mutex objects, semaphore objects, waitable timer objects, kernel mode locks and other synchronization tools. Memory management.

and the concepts of memory organization, memory allocation and page replacement are explained. We explore file systems management, discussing file system drivers and NTFS topics including the Master File Table (MFT), data streams, file compression, file encryption, sparse files, reparse points and mounted volumes. We study input/output management, explaining device drivers, Plug-and-Play, power management, the Windows Driver Model (WDM), input/output processing, I/O request packets (IRPs), synchronous I/O, asynchronous I/O, data transfer techniques, interrupt handling and file cache management. We consider interprocess communication mechanisms including pipes (anonymous and named), mailslots, shared memory, and local and remote procedure calls. Microsoft's Component Object Model (COM) is overviewed. We explain drag-and-drop and compound documents. We discuss networking capabilities, including network input/output, network driver architecture, network protocols (IPX, SPX, NetBEUI, NetBIOS over TCP/IP, WinHTTP, WinINet and Winsock 2. Network services topics including Active Directory, Lightweight Directory Access Protocol (LDAP) and Remote Access Service (RAS) are discussed. We overview Microsoft's new .NET technology which is replacing DCOM. We examine scalability, considering both symmetric multiprocessing (SMP) and Windows XP Embedded. The chapter concludes with a discussion of security topics, including authentication, authorization, Internet Connection Firewall (ICF) and other security features.

Whew! Well, that completes the Tour of the Book. There is much here for operating systems students and professionals.

Ancillary Package for Operating Systems, 3/e

Operating Systems, 3/e has extensive ancillary materials for instructors. The Instructor's Resource CD (IRCD) contains solutions to the vast majority of the end-of-chapter exercises. This CD is available only to instructors through their Prentice Hall representatives. Visit vig.prenhall.com/relocator to locate your Prentice Hall representative. **[NOTE: Please do not write to us requesting the instructor's CD. Distribution of this CD is limited strictly to college professors teaching from the book. Instructors may obtain the solutions manual only from their Prentice Hall representatives.]** The ancillaries for the book also include a Test Item File of multiple-choice questions. In addition, we provide PowerPoint lecture notes containing all figures, illustrations, diagrams and code in the text and bullet points that summarize key portions of the text. Instructors can customize the slides. The PowerPoint® slides are downloadable from www.deitel.com and as part of Prentice Hall's companion Web site (www.prenhall.com/deitel) for *Operating Systems, 3/e*, which offers resources for both instructors and students. For instructors, the Companion Web site includes a Syllabus Manager, which helps instructors plan courses interactively and create online syllabi.

Students also benefit from the functionality of the Companion Web site. Book-specific resources for students include:

- Customizable PowerPoint lecture notes
- Source code for the Java programs

Chapter-specific resources available for students include:

- Chapter objectives
- Outline
- Highlights (e.g., chapter summary)
- Web links

Acknowledgments

One of the great pleasures of writing a textbook is acknowledging the efforts of many people whose names may not appear on the cover, but whose hard work, cooperation, friendship and understanding were crucial to the production of the book. Many of our colleagues at Deitel & Associates, Inc. devoted long hours to this book and its ancillaries: Jeff Listfield, Su Zhang, Abbey Deitel, Barbara Deitel and Christi Kelsey

We would like to include a special note of thanks to Ben Wiedermann for organizing the effort to revise the material in the second edition of this book. He reviewed and updated much of the existing writing and diagrams. He also organized and directed a team of Deitel interns who researched the latest developments in Operating Systems for inclusion in this new edition. Ben currently is pursuing a Ph.D. in Computer Science at the University of Texas, Austin.

We would also like to thank the participants in the Deitel & Associates, Inc., College Internship Program who worked on the content of this book and its ancillary package: Jonathan Goldstein (Cornell University), Lucas Ballard (Johns Hopkins University), Tim Christensen (Boston College), John Paul Casiello (Northeastern University), Mira Meyerovich (Ph.D. candidate at Brown University), Andrew Yang (Carnegie Mellon University), Kathryn Ruigh (Boston College), Chris Gould (University of Massachusetts at Lowell), Marc Manara (Harvard University), Jim O'Leary (Rensselaer Polytechnic Institute), Gene Pang (Cornell University) and Randy Xu (Harvard University).

We would like to acknowledge Howard Berkowitz (formerly of the Corporation for Open Systems International) for co-authoring Chapter 16, Distributed Computing, in the second edition of this text. His work was the basis for our expanded, enhanced and updated treatment of networking and distributed computing in Chapters 16 through 18 of *Operating Systems, 3/e*.

We are fortunate to have worked on this project with the talented and dedicated team of publishing professionals at Prentice Hall. We especially appreciate the extraordinary efforts of our Computer Science Editor, Kate Hargett and her boss and our mentor in publishing—Marcia Horton, Editorial Director of Prentice-Hall's Engineering and Computer Science Division. Vince O'Brien, Tom Manshreck, John Lovell and Chirag Thakkar did a marvelous job managing the production of the book. Sarah Parker managed the publication of the book's ancillary package.

We would like to thank the design team that created a completely new look and feel for *Operating Systems, 3/e*—Carole Anson, Paul Belfanti, Geoffrey Cassar and John Root. We would also like to thank the talented creative team at Artworks, including Kathryn Anderson, Jay McElroy, Ronda Whitson, Patricia Burns, Matt Haas, Xiaohong Zhu, Dan Missildine, Chad Baker, Sean Hogan, Audrey Simonetti, Mark Landis, Royce Copenheaver, Stacy Smith, Scott Wieber, Pam Taylor, Anna Whalen, Cathy, Shelly, Ken Mooney, Tim Nguyen, Carl Smith, Jo Thompson, Helenita Zeigler and Russ Crenshaw.

The most important acknowledgment is to the thousands of authors represented in the Recommended Readings and Works Cited sections in the chapters; their research papers, articles and books have provided the diversity of interesting material that makes operating systems such a fascinating area.

We wish to acknowledge the efforts of our *Third Edition* reviewers and to give a special note of thanks to Carole Snyder and Jennifer Cappello of Prentice Hall who managed this extraordinary review effort. Adhering to a tight time schedule, these reviewers scrutinized the text, providing countless suggestions for improving the accuracy and completeness of the presentation. It is a privilege to have the guidance of such talented and busy professionals.

Third Edition reviewers:

Tigran Aivazian, Veritas Software, Ltd.
 Jens Axboe, SUSE Labs
 Dibyendu Bakshi, Scientific Technologies Corporation
 Columbus Brown, IBM
 Fabian Bustamante, Northwestern University
 Brian Catlin, Azius Developer Training
 Stuart Cedrone, Hewlett-Packard
 Randy Chow, University of Florida
 Alan Cox, Red Hat UK
 Matthew Dobson, Independent Consultant
 Ulrich Drepper, Red Hat, Inc.
 Alessio Gaspar, University of South Florida, Lakeland Campus
 Allison Gehrke, Colorado Technical University
 Michael Green, Central Texas College
 Rob Green, Columbia Data Products, Inc.
 James Huddleston, Independent Consultant
 Scott Kaplan, Amherst College
 Salahuddin Khan, Microsoft
 Robert Love, MontaVista Software, Inc.
 Barry Margolin, Level 3 Communications
 Cliff Mather, Hewlett-Packard
 Jeanna Matthews, Cornell University/Clarkson University
 Manish Mehta, Syntel, India, Ltd.
 Dejan Milojicic, Hewlett-Packard

Euripides Montagne, University of Central Florida
Andrew Morton, Diego, Inc.
Gary Newell, Northern Kentucky University
Bill O'Farrell, IBM
Mike Panetta, Robotic Sciences, Inc./Netplanner Systems, Inc.
Rohan Phillips, Microsoft
Atul Prakash, University of Michigan
Bina Ramamurthy, SUNY Buffalo
Eric Raymond, Thyrus Enterprises
Jeffrey Richter, Wintellect
Sven Schreiber, Platon Data Technology
Wennie Wei Shu, University of New Mexico
David Solomon, David Solomon Expert Seminars
Brandon Taylor, Sun Microsystems
Bob Toxen, Fly-By-Day Consulting, Inc.
Joshua Uziel, Sun Microsystems
Carlos Valcarcel, EinTech, Inc.
Melinda Ward, Hewlett-Packard
Gerard Weatherby, Charles Consulting, LLC. /Rensselaer at Hartford
Xiang Xu, RSA Security, Inc.

***First and Second Edition reviewers and contributors
(and their affiliations at that time):***

Howard Berkowitz, Corporation for Open Systems
Dale A. Brown, The College of Wooster
Steven J. Buroff, AT&T Bell Laboratories
David D. Busch
John H. Carson, George Washington University
Ronald Curtis, Canisius College
Larry K. Flanigan, University of Michigan
Jon Forrest, Sybase
Jay Gadre, Corporation for Open Systems
Carlos M. Gonzalez, George Mason University
Mike Halvorson, Microsoft Press
Wayne Hathaway, Ultra Network Technologies
Christopher T Haynes, Indiana University
Lee Hollaar, University of Utah
William Horst, Corporation for Open Systems
James Johannes, University of Alabama
Ralph Johnson, University of Illinois
Dennis Kafura, Virginia Polytechnical Institute
Herb Klein, Corporation for Open Systems
Michael S. Kogan, IBM
Thomas LeBlanc, University of Rochester

T. F. Leibfried, University of Houston, Clear Lake
David Litwack, Corporation for Open Systems
Mark Measures, Baylor University
Charles Oualline, Jr., East Texas State University
Ravi Sandhu, Ohio State University
Richard Schlichting, University of Arizona
Alan Southerton
John J. Zenor, California State University
James Peterson, University of Texas at Austin
Richard Wexelblatt, Sperry Univac
Paul Ross, Millersville State University
Anthony Lucido, Intercomp
Steve Paris, Prime Computer
Bart Guerreri, DSD Laboratories
Nathan Tobol, Codex Corporation and Chairman, IEEE 802 Local-Area
Networking Subcommittee
Larry Nelson, AVCO Services
Barry Shein, Harvard University
Eliezer Garni, MIT
Anat Gafni, Boston University
Josefina Bondoc, Boston University

Contacting Deitel & Associates

We would sincerely appreciate your comments, criticisms, corrections and suggestions for improving the text. Please address all correspondence to:

deitel@deitel.com

We will respond promptly.

Deitel Buzz® Online E-mail Newsletter

Our free e-mail newsletter, the *DEITEL BUZZ® Online*, includes updates to *Operating Systems 3/e*, commentary on industry trends and developments, links to free articles and resources from our published books and upcoming publications, product-release schedules, errata, challenges, anecdotes, information on our corporate instructor-led training courses and more. To subscribe, visit

www.deitel.com/newsletter/subscribe.html

About Deitel & Associates, Inc.

Deitel & Associates, Inc., is an internationally recognized corporate training and content-creation organization specializing in Internet/World Wide Web software technology, e-business/e-commerce software technology, object technology, operating systems and computer programming languages education. The company pro-

vides instructor-led courses on Internet and Web programming, wireless Internet programming, object technology, and major programming languages and platforms, including C, C++, Visual C++® .NET, Visual Basic® .NET, C#, Java, J2EE, XML, Perl, Python and more. The founders of Deitel & Associates, Inc., are Dr. Harvey M. Deitel and Paul J. Deitel. The company's clients include many of the world's largest computer companies, government agencies, branches of the military and business organizations. Through its 27-year publishing partnership with Prentice Hall, Deitel & Associates, Inc., publishes leading-edge programming textbooks, professional books, interactive CD-based multimedia *Cyber Classrooms*, *Complete Training Courses*, Web-based training courses and course management systems e-content for popular CMSs such as WebCT™, Blackboard™ and CourseCompassSM.

To learn more about Deitel & Associates, Inc., its publications and its corporate on-site training curriculum, see the last few pages of this book or visit:

www.deitel.com

Individuals wishing to purchase Deitel books, *Cyber Classrooms*, *Complete Training Courses* and Web-based training courses can do so through bookstores, online booksellers and the following Web sites:

www.deitel.com

www.prenhall.com/deitel

www.InformIT.com/deitel

www.InformIT.com/cyberclassrooms

Bulk orders by corporations and academic institutions should be placed directly with Prentice Hall. See www.prenhall.com/deitel for worldwide ordering instructions and to find your local Prentice Hall sales representative.

Welcome to the exciting world of operating systems. We sincerely hope you enjoy learning with this book.

Dr. Harvey M. Deitel

Paul J. Deitel

David R. Choffnes

About the Authors

Dr. Harvey M. Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has 42 years experience in the computing field, including extensive industry and academic experience. Dr. Deitel studied operating systems at the Massachusetts Institute of Technology where he earned B.S. and M.S. degrees. Dr. Deitel earned his Ph.D. from Boston University. While an undergraduate and master's candidate at MIT, he worked on the pioneering virtual-memory operating-systems projects at IBM (OS/360 and TSS/360) and MIT (Multics) that developed techniques now widely implemented in systems including UNIX, Linux and Windows

XP. He taught operating systems for 20 years in academia and industry. He earned tenure and served as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., with his son, Paul J. Deitel. He and Paul are the co-authors of several dozen books and multimedia packages and they are writing many more. With translations published in Japanese, German, Russian, Spanish, Traditional Chinese, Simplified Chinese, Korean, French, Polish, Italian, Portuguese, Greek, Urdu and Turkish, the Deitels' texts have earned international recognition. Dr. Deitel has delivered professional seminars to major corporations, government organizations and the military.

Paul J. Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., is a graduate of the Massachusetts Institute of Technology's Sloan School of Management, where he studied information technology and operating systems. Through Deitel & Associates, Inc., he has delivered hundreds of courses to Fortune 500, academic, government and military clients, and many other organizations. He has lectured for the Boston Chapter of the Association for Computing Machinery. He and his father, Dr. Harvey M. Deitel, are the world's best-selling computer science textbook authors.

David R. Choffnes is a graduate of Amherst College, where he earned degrees in Physics and French while pursuing advanced topics in Computer Science. His research interests include the fields of operating systems, computer architecture, computational biology and molecular computing. David has contributed to other Deitel publications including *Simply Java™ Programming* and *Simply Visual Basic® .NET*.

Introduction to Hardware, Software and Operating Systems

Intelligence ...is the faculty of making artificial objects, especilly tolls to make tools.

—Henri Bergson—

Part 1